

Tecnologías Específicas en la Ingeniería Informática
Práctica Bloque II: Virtualización y Contenedores

Profesores:

- **Pantaleone Nespoli**
- **Noelia María Pérez Palma**

Autor:

- **Rafael Guillén García:**
 - **Grupo: 3.3.**
 - **DNI: [REDACTED].**
 - **Correo: rafael.guilleng@um.es**

Fecha de Entrega:

- **27 de marzo de 2026**

Índice

1.	Parte 1 – Virtualización con VirtualBox, KVM y Vagrant.	3
•	VirtualBox	3
1.	Ejercicio 1.....	3
2.	Ejercicio 2.....	3
3.	Ejercicio 3.....	16
4.	Ejercicio 4.....	17
5.	Ejercicio 5.....	17
6.	Ejercicio 6.....	20
7.	Ejercicio 7.....	20
•	KVM/LIBVIRT/VIRSH	23
1.	Ejercicio 1.....	23
2.	Ejercicio 2.....	23
3.	Ejercicio 3.....	24
4.	Ejercicio 4.....	24
5.	Ejercicio 5.....	31
•	Vagrant.....	32
1.	Ejercicio 1.....	32
2.	Ejercicio 2.....	34
3.	Ejercicio 3.....	34
4.	Ejercicio 4.....	35
2.	Parte 2 – Contenedores con Docker	37
•	Primer contenedor.....	37
•	Segundo Contenedor	38
•	Tercer Contenedor	39
•	Docker-compose	40

1. Parte 1 – Virtualización con VirtualBox, KVM y Vagrant.

• VirtualBox

1. Ejercicio 1. Crear una red Host-Only vboxnet0 con:

- Red: 192.168.56.0/24.
- IP del host en esa red: 192.168.56.1.

La red Host-Only es una red privada entre el host y las máquinas virtuales, es decir, permite la comunicación entre el host y las máquinas virtuales conectadas a dicha red sin que intervenga directamente la red física externa.

Para realizar este ejercicio se utiliza VBoxManage, que es la herramienta de línea de comandos de VirtualBox. En esta herramienta, se utiliza el comando *hostonlyif* el cual sirve para gestionar interfaces de red de tipo Host-Only. Además, se utilizará el subcomando *create*, que permite crear una nueva interfaz de este tipo. También, se utilizará el subcomando *ipconfig* con las opciones *--ip* y *--netmask* para asignarle su configuración.

Entonces, primero se crea la interfaz vboxnet0 mediante el comando *VBoxManage hostonlyif create*:

```
(base) rafa@portatilrafa:~/TEII/bloque2$ VBoxManage hostonlyif create  
0%...10%...20%...30%...40%...50%...60%...70%...80%...90%...100%  
Interface 'vboxnet0' was successfully created
```

Ahora, se configura la interfaz con la dirección IP 192.168.56.1 y máscara 255.255.255.0 (/24), que corresponde a la red 192.168.56.0/24, con el comando *VBoxManage hostonlyif ipconfig vboxnet0 --ip=192.168.56.1 --netmask=255.255.255.0*:

```
(base) rafa@portatilrafa:~/TEII/bloque2$ VBoxManage hostonlyif ipconfig vboxnet0 --ip=192.168.56.1 --netmask=255.255.255.0
```

De esta forma, vboxnet0 queda configurada dentro de la red 192.168.56.0/24.

2. Ejercicio 2. Crear una VM llamada SRV con:

- 2GB RAM, 2 CPUs, 25 GB disco SATA (VDI)
- Ubuntu Server 24.04.
- Adaptador 1: NAT.
- Adaptador 2: Host-Only (vboxnet0) con IP fija 192.168.56.10.

Para realizar este ejercicio, se utiliza el comando *createvm* para crear y registrar la nueva máquina virtual. Después, con el comando *modifyvm* se ajustan sus recursos hardware y la configuración de red mediante las opciones *--memory* (para indicar la cantidad de RAM en MB), *--cpus* (para indicar el número de CPUs), *--nic1*, *--nic2* y *--hostonlyadapter2*. Estas dos últimas opciones conectan el segundo adaptador a la red vboxnet0, y el modo NAT se usa para que la máquina virtual tenga acceso al exterior a través del host.

Para el almacenamiento, primero se crea un disco virtual con el comando *createhd*. Después, con *storagectl* se añaden los controladores virtuales de almacenamiento a la máquina, y con *storageattach* se asocia el disco VDI al controlador SATA y la imagen ISO al controlador IDE. Así, la máquina queda lista para poder arrancar la ISO e instalar Ubuntu.

La dirección IP no se pone directamente con VBoxManage, sino que se debe realizar en Ubuntu.

Entonces, primero se crea la máquina virtual SRV y se registra en virtualbox mediante el comando: *VBoxManage createvm --name SRV --ostype Ubuntu_64 --register*

```
(base) rafa@portatilrafa:~$ VBoxManage createvm --name SRV --ostype Ubuntu_64 --register
Virtual machine 'SRV' is created and registered.
UUID: 7b9da4ef-9392-4d54-985e-79d7af69b904
Settings file: '/home/rafa/VirtualBox VMs/SRV/SRV.vbox'
```

Ahora, se configuran la memoria, las CPUs y los adaptadores de red: el primero en modo NAT y el segundo en modo Host-Only conectado a vboxnet0. Para ello, se usa el comando *VBoxManage modifyvm SRV --memory 2048 --cpus 2 --nic1 nat --nic2 hostonly --hostonlyadapter2 vboxnet0*

```
(base) rafa@portatilrafa:~$ VBoxManage modifyvm SRV --memory 2048 --cpus 2 --nic1 nat --nic2 hostonly --hostonlyadapter2 vboxnet0
```

Tras esto, se crea el disco virtual VDI de 25GB con el comando: *VBoxManage createhd --filename /home/rafa/TEII/bloque2/disco_ubuntu.vdi --size 25625*

```
(base) rafa@portatilrafa:~$ VBoxManage createhd --filename /home/rafa/TEII/bloque2/disco_ubuntu.vdi --size 25625
0%...10%...20%...30%...40%...50%...60%...70%...80%...90%...100%
Medium created. UUID: a42ae86a-666c-4689-8747-4ad7f94722b3
```

Ahora, se añade a un controlador SATA, introduciendo la siguiente secuencia de comandos:

1º *VBoxManage storagectl SRV --name "SATA Controller" --add sata --controller IntelAhci*

2º *VBoxManage storageattach SRV --storagectl "SATA Controller" --type hdd --port 0 --device 0 --medium /home/rafa/TEII/bloque2/disco_ubuntu.vdi*

```
(base) rafa@portatilrafa:~$ VBoxManage storagectl SRV --name "SATA Controller" --add sata --controller IntelAhci
(base) rafa@portatilrafa:~$ VBoxManage storageattach SRV --storagectl "SATA Controller" --type hdd --port 0 --device 0 --medium /home/rafa/TEII/bloque2/disco_ubuntu.vdi
```

Lo siguiente es añadir la unidad de DVD virtual y se monta la imagen ISO de Ubuntu Server 24.04 para poder instalar el sistema operativo. Para ello se usan los siguientes comandos:

1º *VBoxManage storagectl SRV --name "IDE Controller" --add ide --controller PIIX4*

2º *VBoxManage storageattach SRV --storagectl "IDE Controller" --type DVDDrive --port 1 --device 0 --medium /home/rafa/TEII/bloque2/ubuntu-24.04.3-live-server-amd64.iso*

```
(base) rafa@portatilrafa:~$ VBoxManage storagectl SRV --name "IDE Controller" --add ide --controller PIIX4
(base) rafa@portatilrafa:~$ VBoxManage storageattach SRV --storagectl "IDE Controller" --type DVDDrive --port 1 --device 0 --medium /home/rafa/TEII/bloque2/ubuntu-24.04.3-live-server-amd64.iso
```

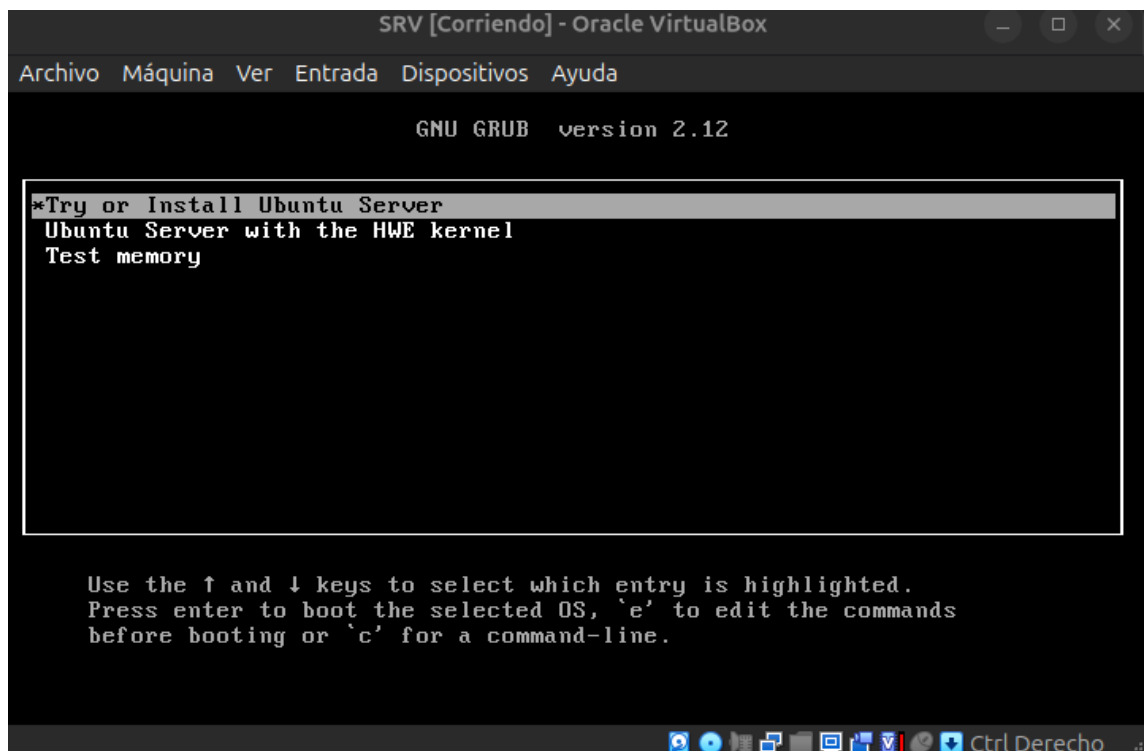
Para comprobar que la máquina se ha configurado correctamente, se utiliza el comando *VBoxManage showvminfo SRV*:

```
(base) rafa@portatilrafa:~$ VBoxManage showvminfo SRV
Name: SRV
Encryption: disabled
Groups: /
Platform Architecture: x86
Guest OS: Ubuntu (64-bit)
UUID: 7b9da4ef-9392-4d54-985e-79d7af69b904
Config file: /home/rafa/VirtualBox VMs/SRV/SRV.vbox
Snapshot folder: /home/rafa/VirtualBox VMs/SRV/Snapshots
Log folder: /home/rafa/VirtualBox VMs/SRV/Logs
Hardware UUID: 7b9da4ef-9392-4d54-985e-79d7af69b904
Memory size: 2048MB
Page Fusion: disabled
VRAM size: 8MB
CPU exec cap: 100%
CPUProfile: host
Chipset: piix3
Firmware: BIOS
Number of CPUs: 2
```

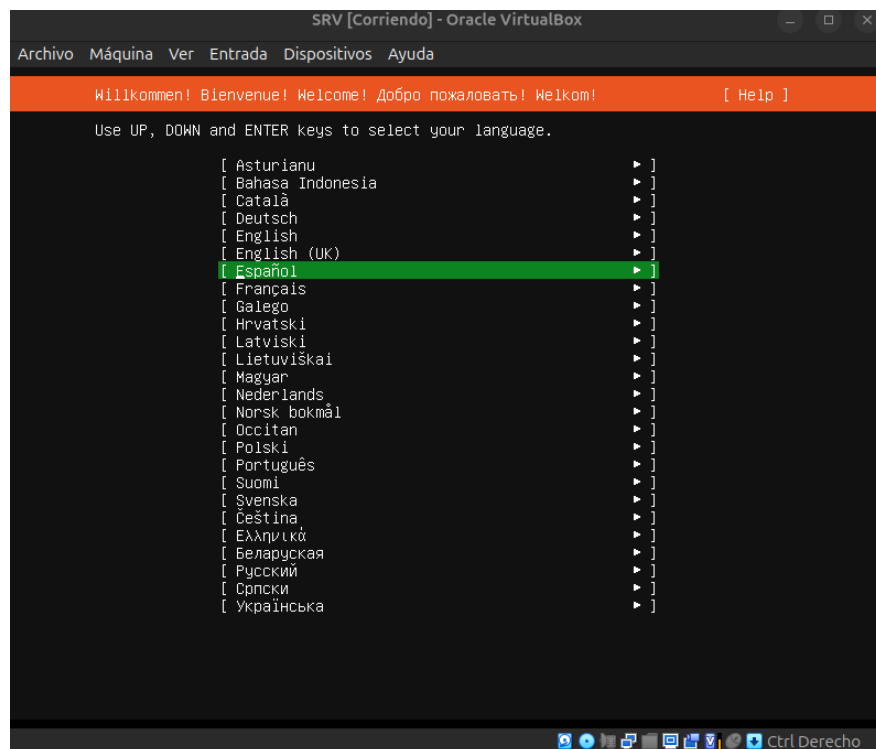
Ahora, una vez preparada la máquina, hay que iniciarla para realizar la instalación, con el comando *VBoxManage startvm SRV*

```
(base) rafa@portatilrafa:~$ VBoxManage startvm SRV
Waiting for VM "SRV" to power on...
VM "SRV" has been successfully started.
```

Una vez se inicie, se selecciona la opción “Try or Install Ubuntu Server”, como se puede apreciar en la siguiente imagen:



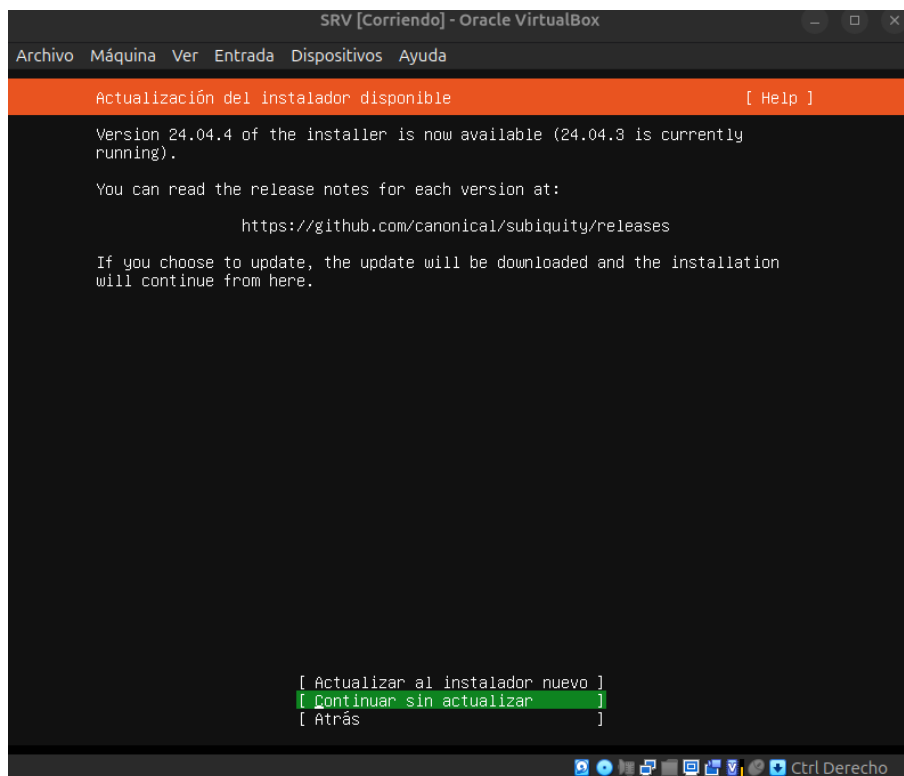
Tras esperar unos segundos, se mostrará lo siguiente:



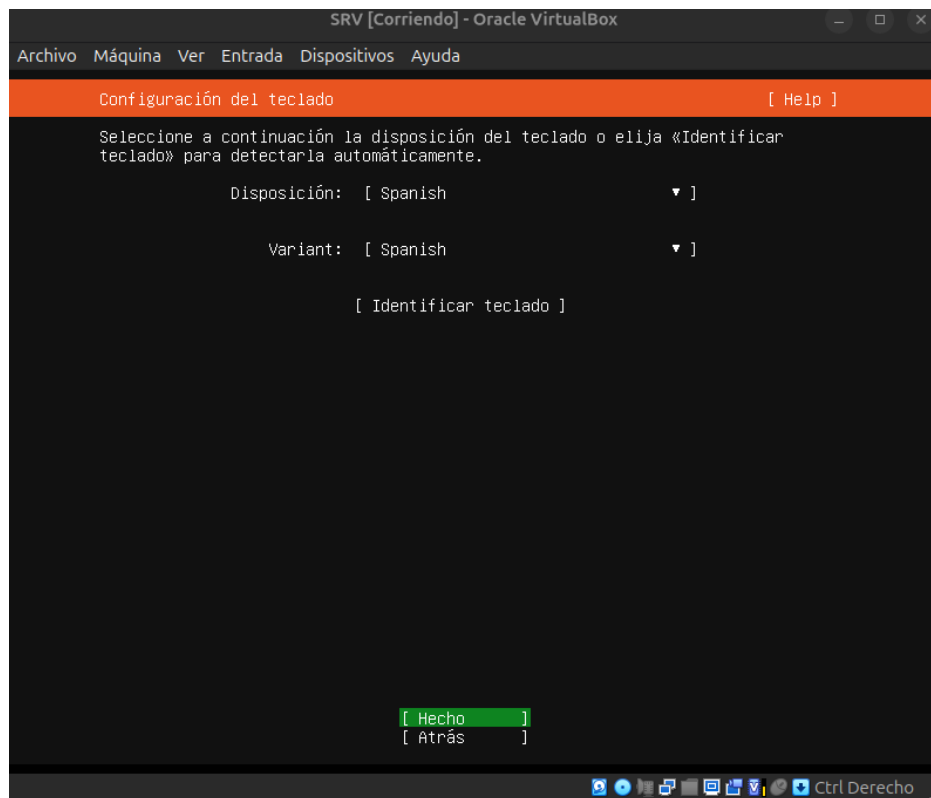
Es decir, empezará la configuración de Ubuntu y se podrá asignar la dirección IP que exige este ejercicio. La configuración que se realizará será la siguiente:

1º Seleccionar el idioma “Español” (tal y como se puede ver en la imagen anterior)

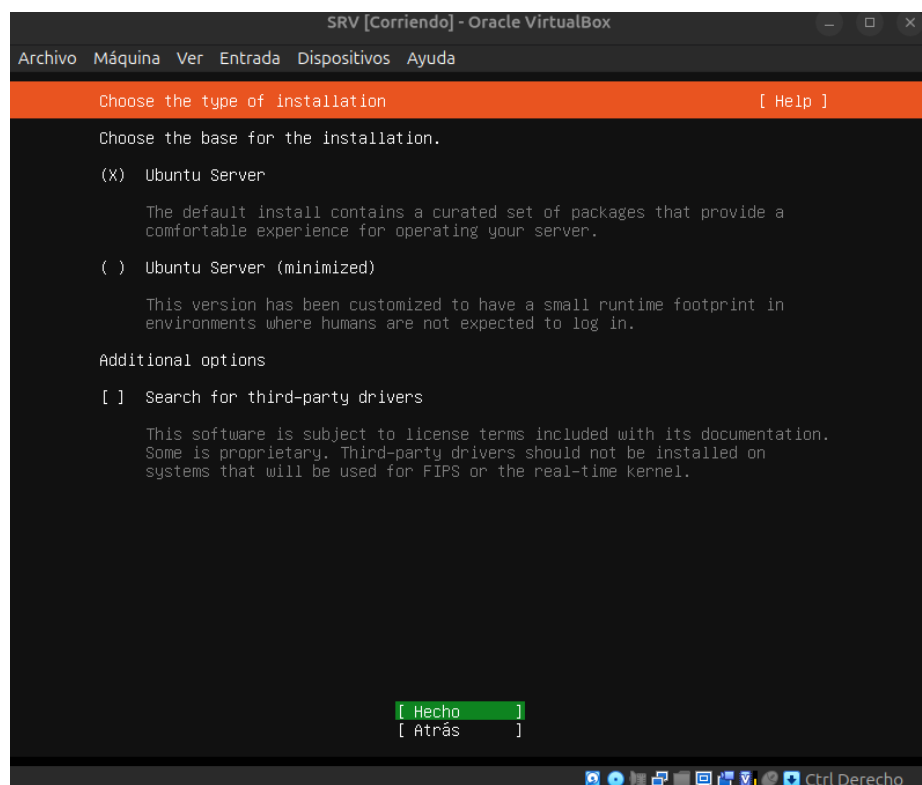
2º Seleccionar “Continuar sin actualizar”, como se ve en la siguiente imagen:



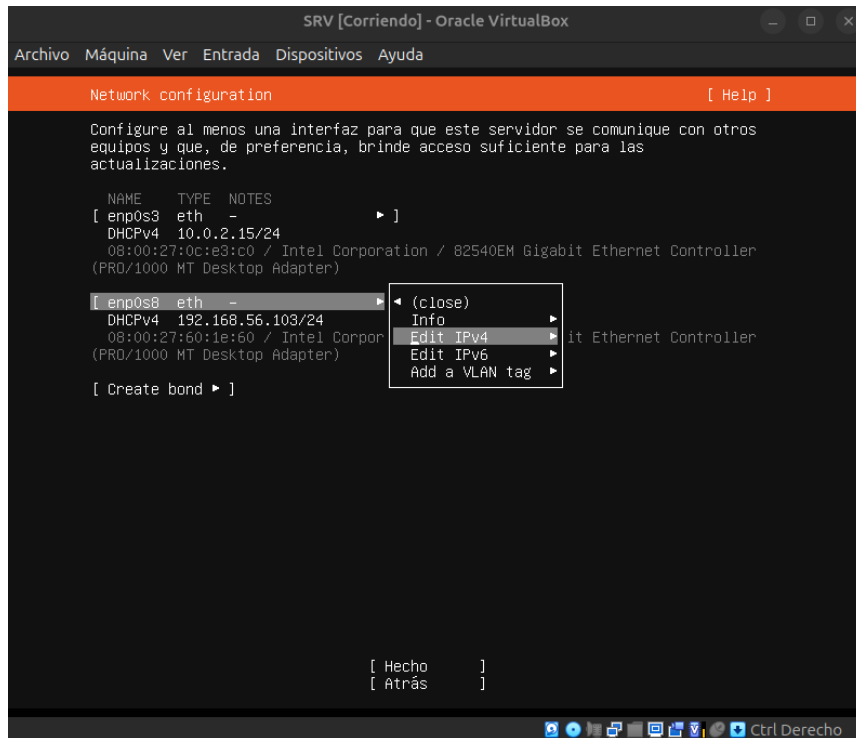
3º Seleccionar “Hecho”, sin cambiar las demás opciones, como se ve en la siguiente imagen:



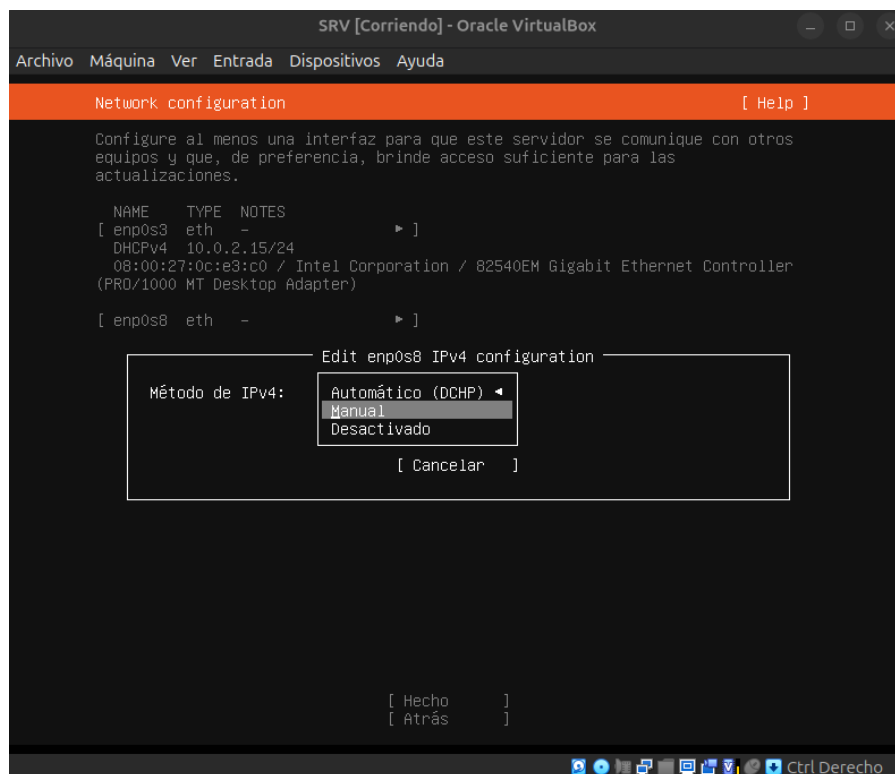
4º Seleccionar “Ubuntu Server” y después seleccionar “Hecho”, como se aprecia en la siguiente imagen:



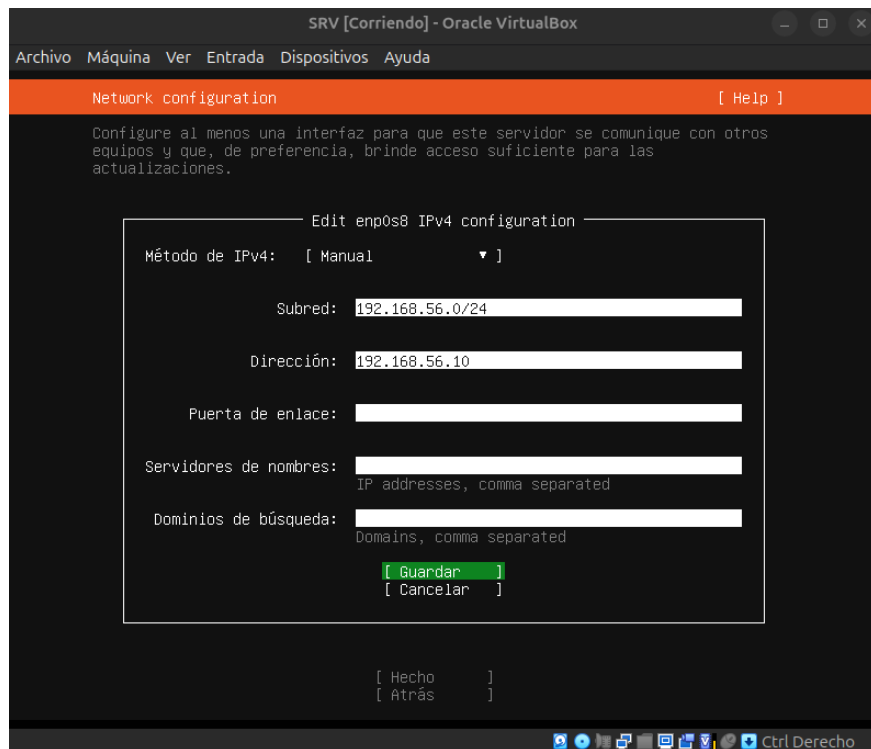
5º Seleccionar la opción “enp0s8” y tras ello “Edit IPv4”, tal y como se ve en la siguiente imagen:



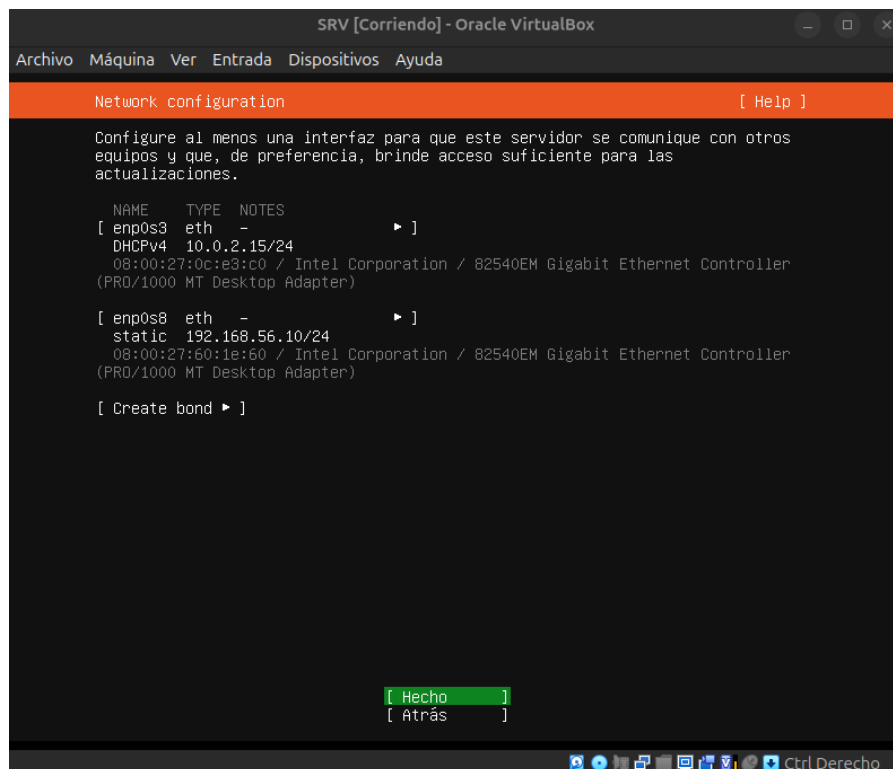
6º En la nueva ventana, seleccionar el modo “Automático (DHCP)” y cambiarlo por “Manual”, como se puede observar:



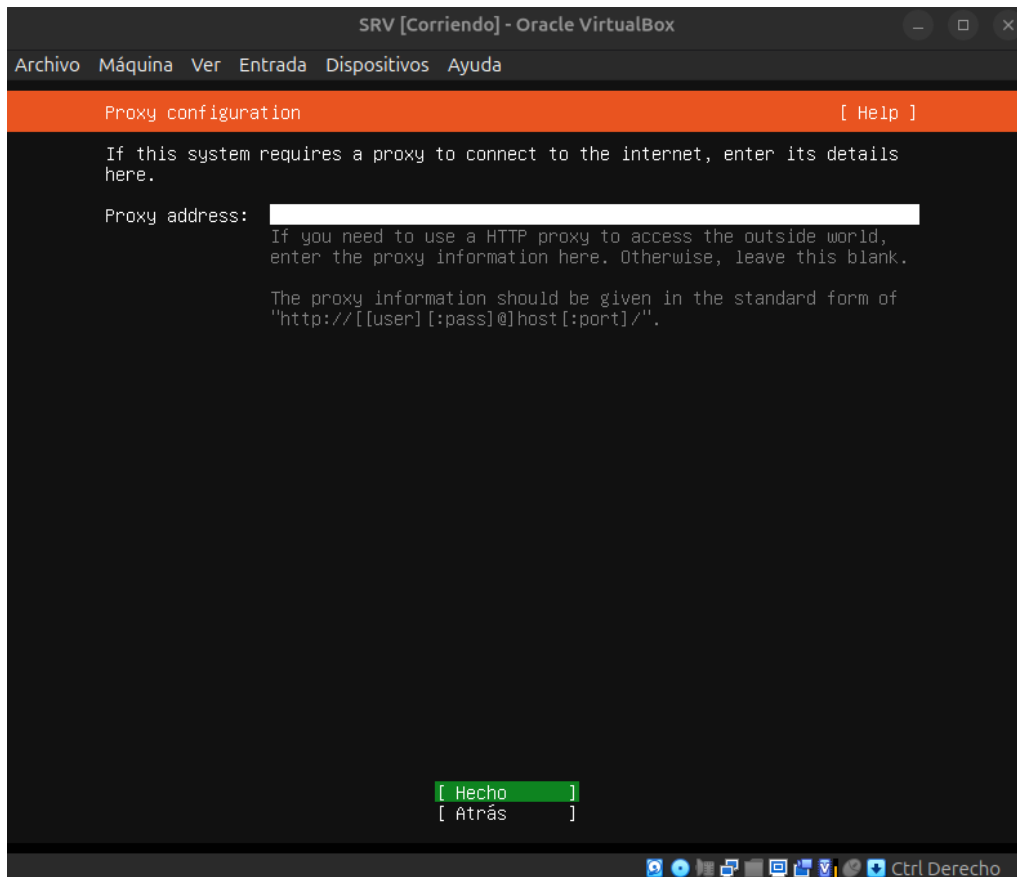
7º Indicar la subred y la dirección (que serán 192.168.56.0/24 y 192.168.56.10) y después seleccionar “Guardar”, tal y como se ve en la siguiente imagen:



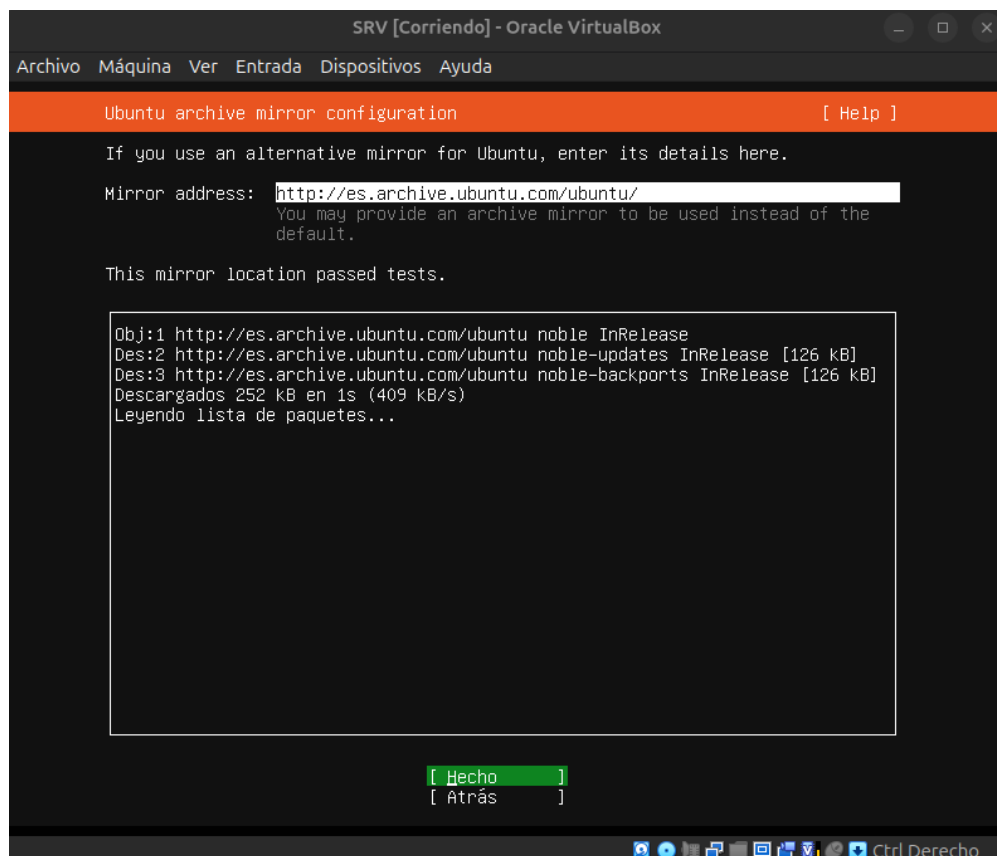
8º Observar que se ha actualizado la interfaz “enp0s8” a la configuración realizada y seleccionar “Hecho”:



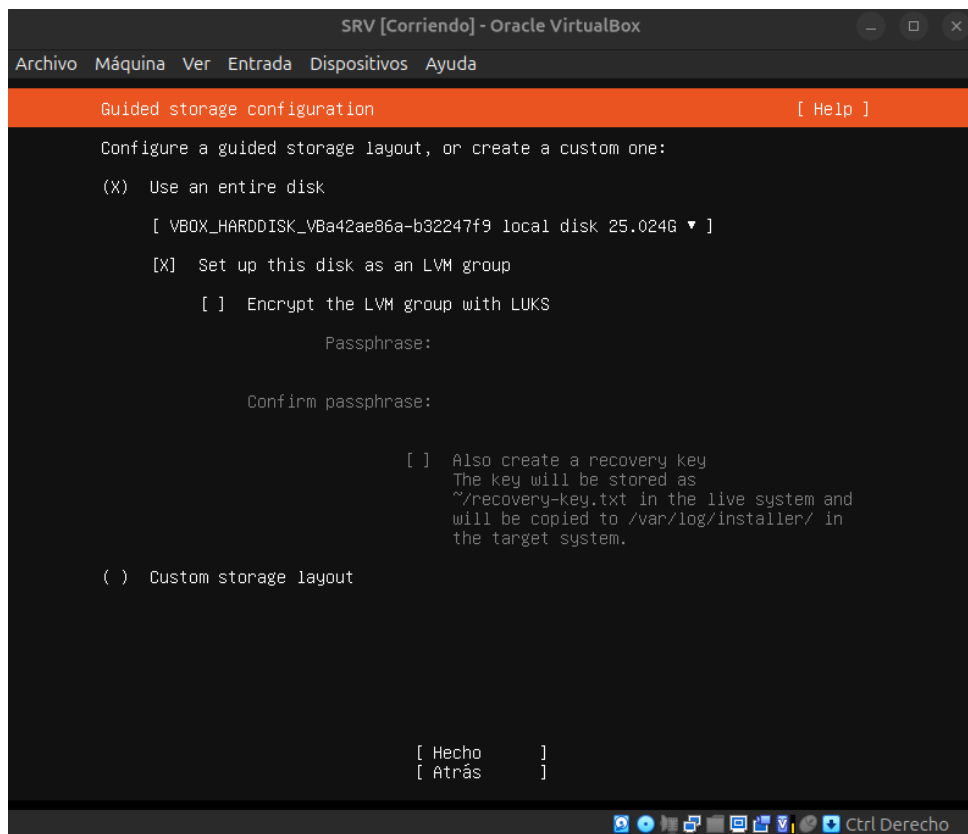
9º Seleccionar “Hecho” sin cambiar nada:



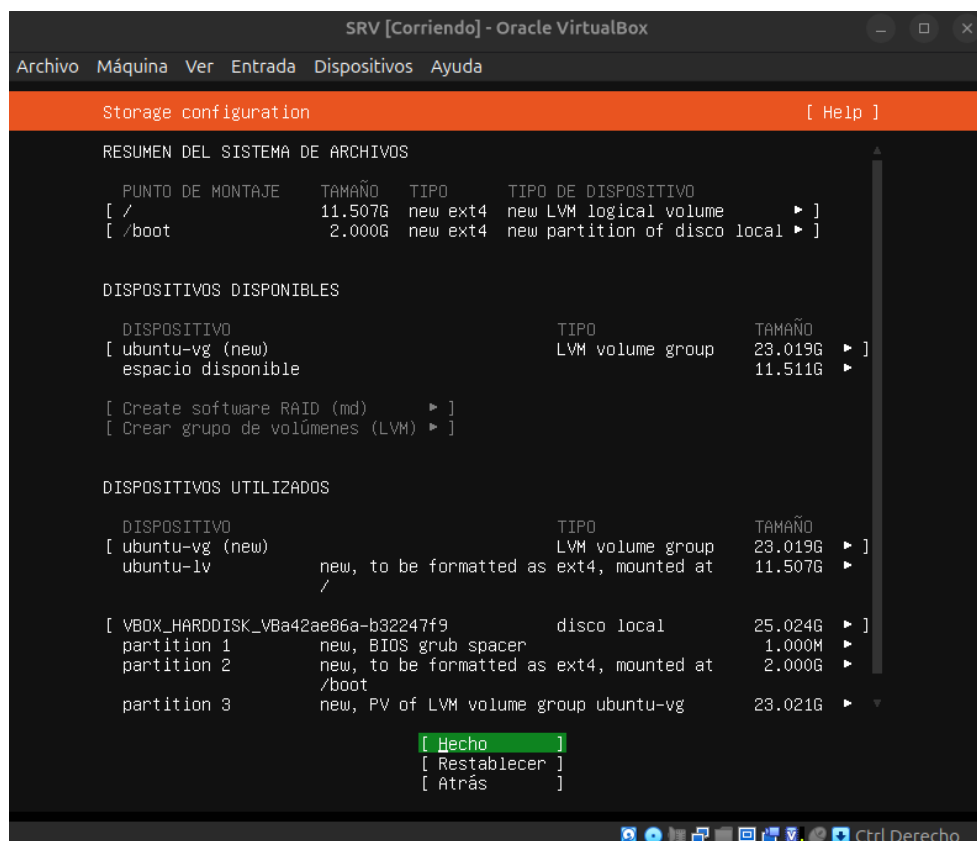
10º Seleccionar “Hecho” sin cambiar nada:



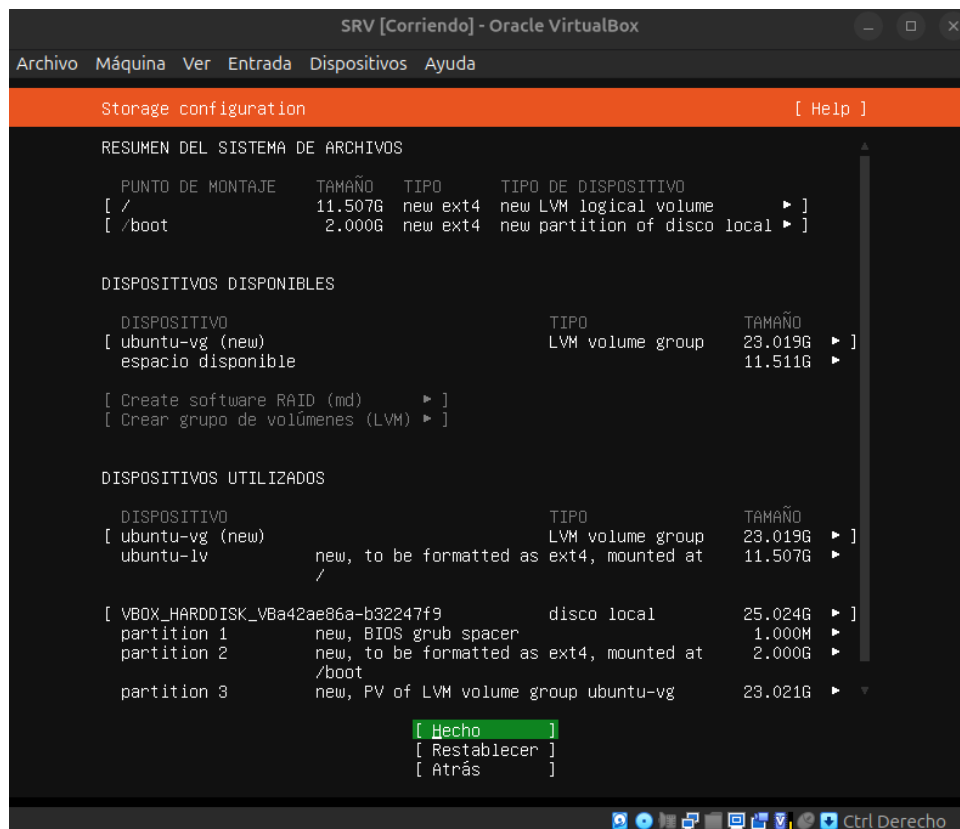
11º Seleccionar “Hecho” sin cambiar nada:



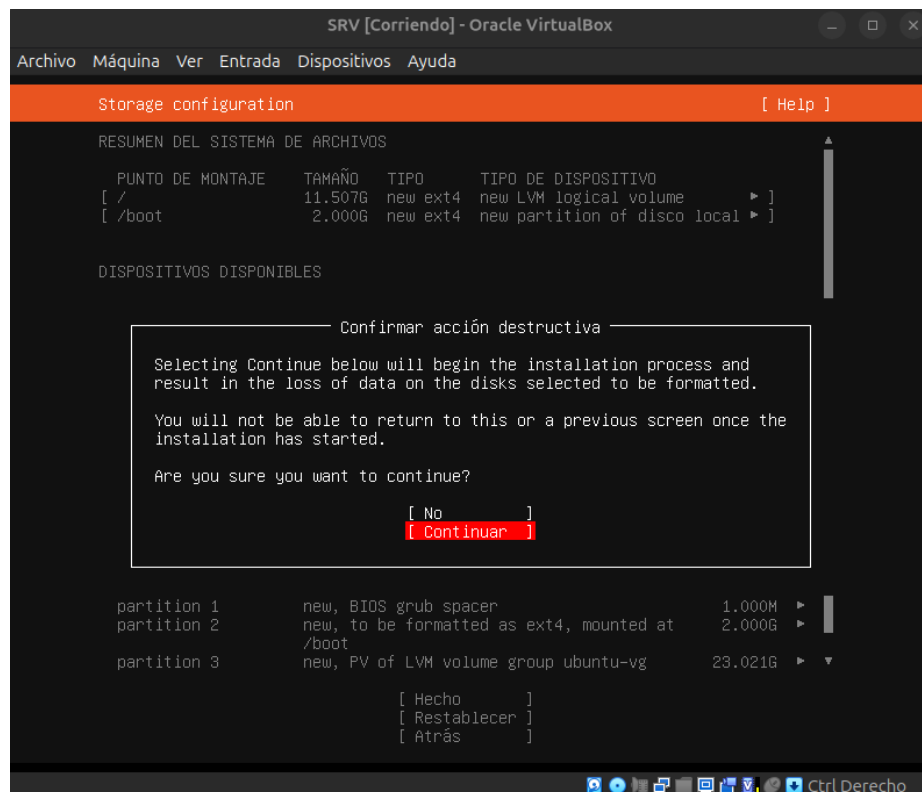
12º Seleccionar “Hecho” sin cambiar nada:



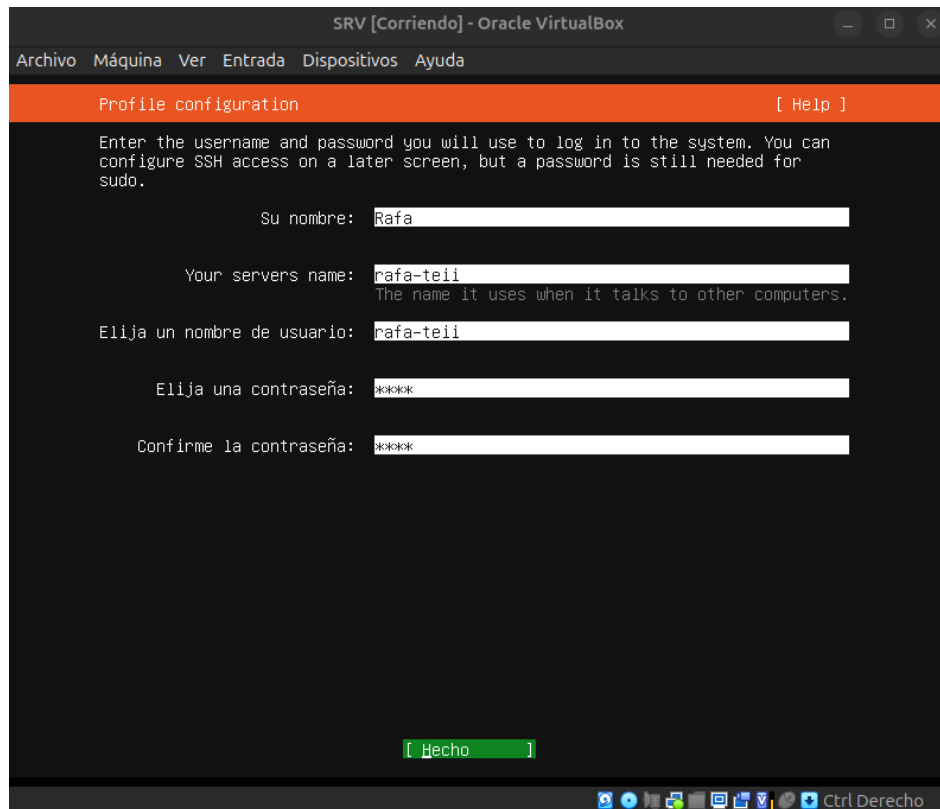
13º Seleccionar “Hecho” sin cambiar nada:



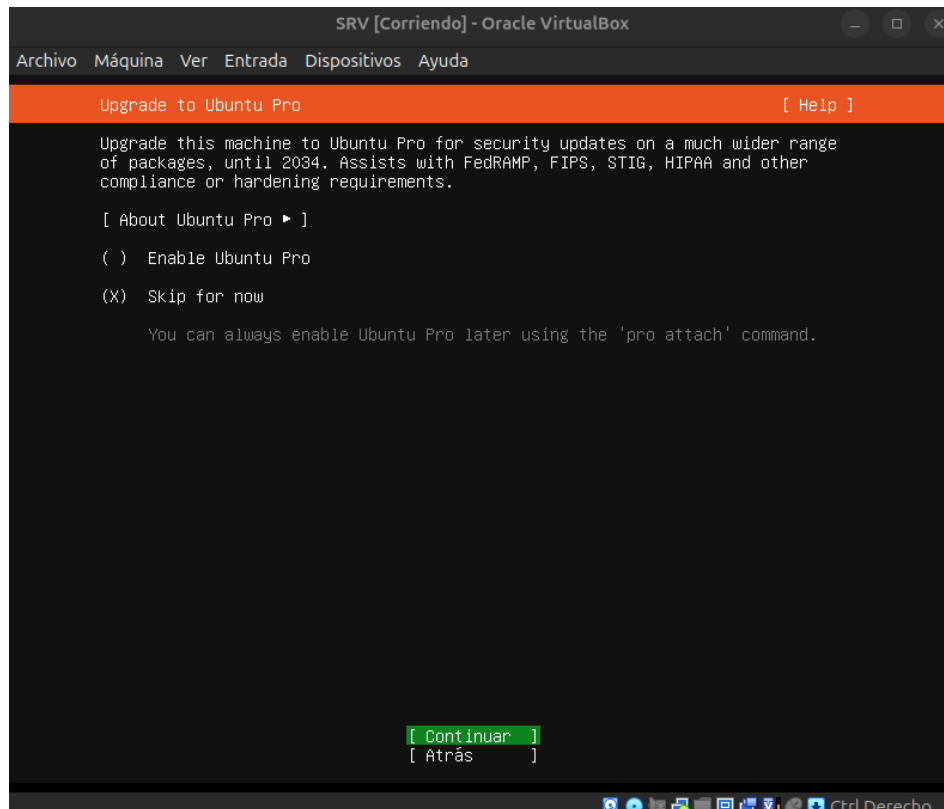
14º Seleccionar “Continuar” en la ventana que se muestra tras el paso anterior:



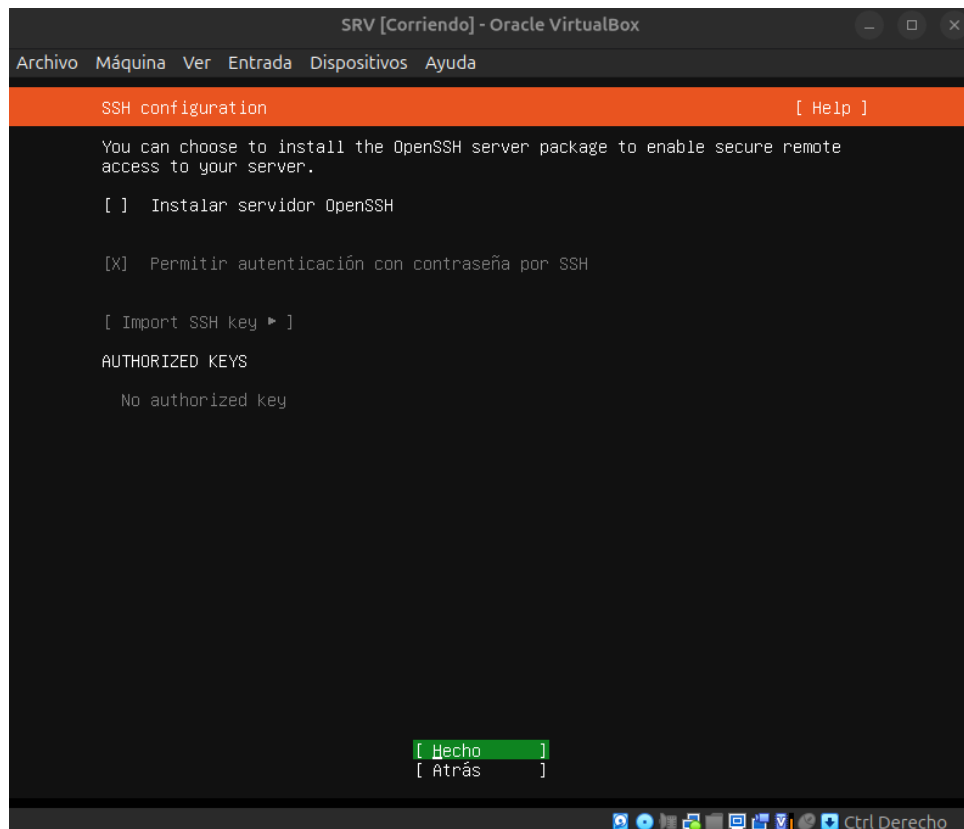
15° Indicar el nombre, nombre del server, nombre de usuario y una contraseña, y seleccionar “Hecho”, como en el ejemplo de la siguiente imagen:



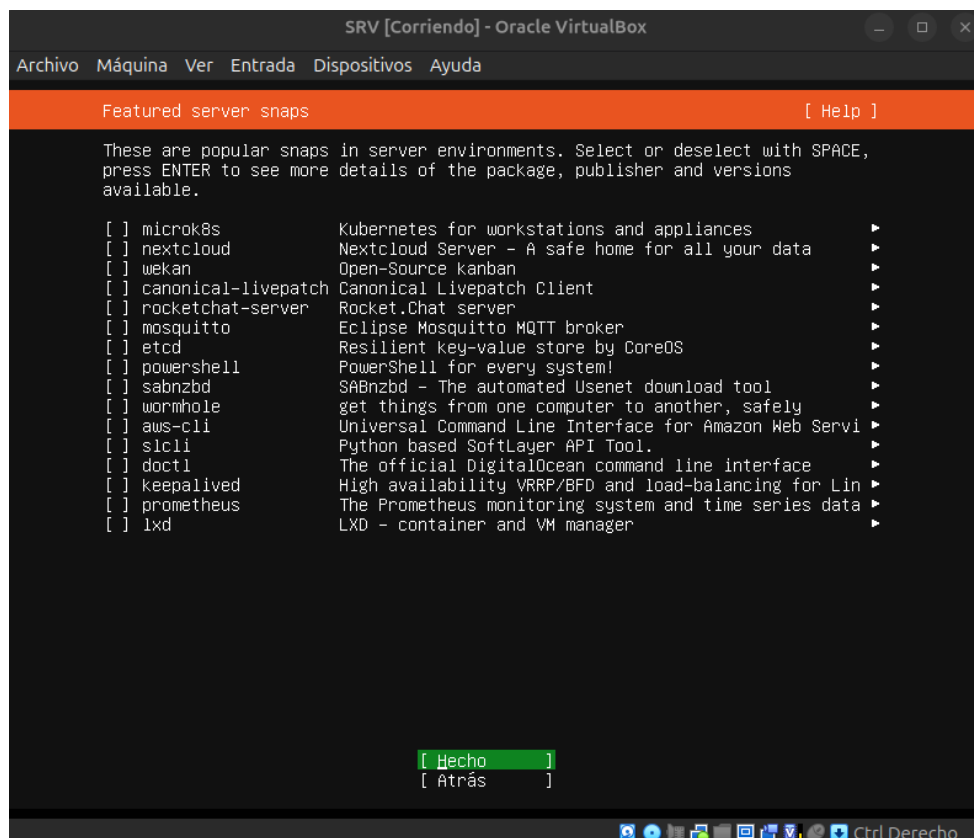
16° Marcar la opción “Skip for now” y seleccionar “Continuar”:



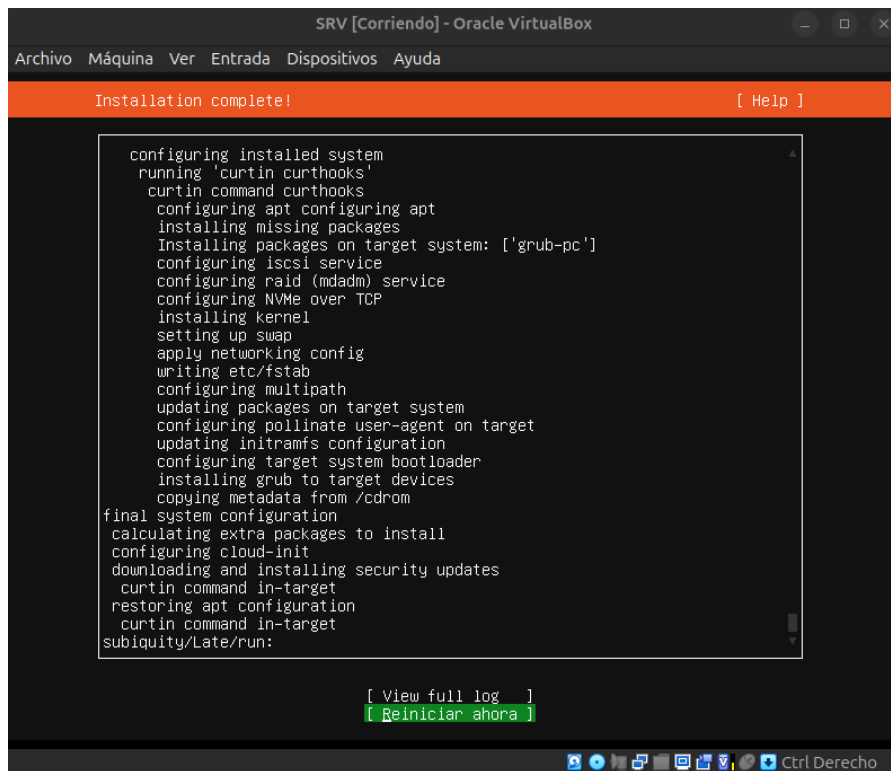
17º No cambiar nada y seleccionar “Hecho”:



18º No marcar ninguna opción y seleccionar “Hecho”:



19º Esperar a que finalice la instalación del sistema y seleccionar “Reiniciar ahora”:

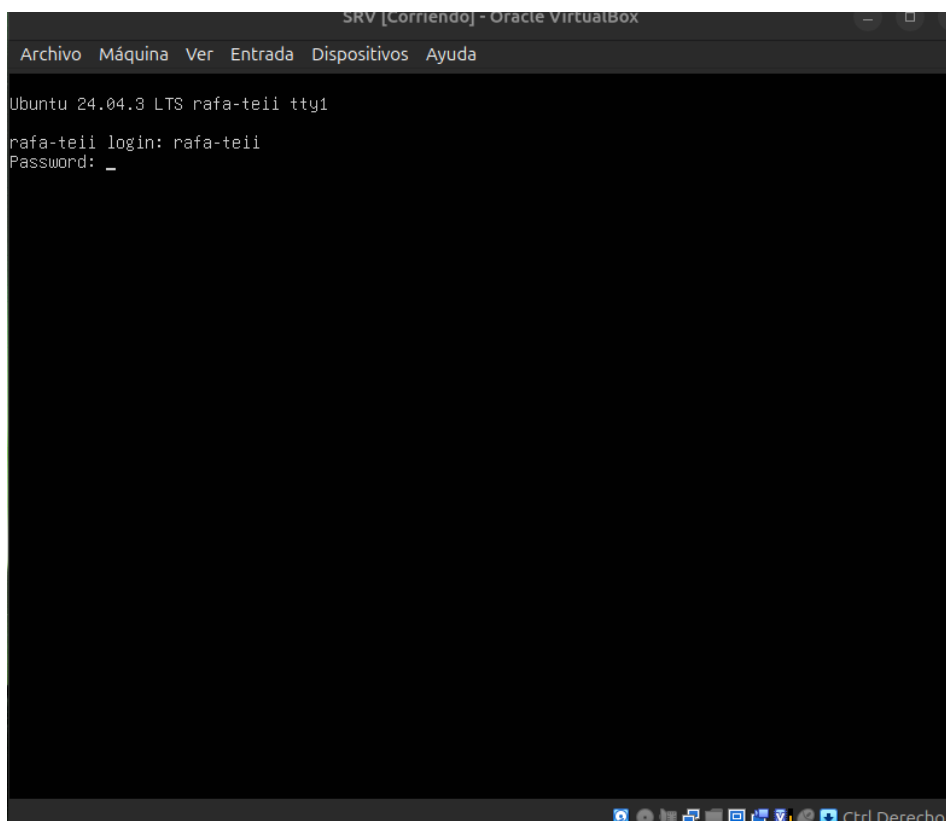


The screenshot shows the 'SRV [Corriendo] - Oracle VirtualBox' window. At the top, a menu bar includes 'Archivo', 'Máquina', 'Ver', 'Entrada', 'Dispositivos', and 'Ayuda'. Below the menu is an orange banner with the text 'Installation complete!' and a '[Help]' link. The main area is a terminal window displaying the following text:

```
configuring installed system
running 'curtin curthooks'
curtin command curthooks
  configuring apt
  configuring apt
  installing missing packages
  Installing packages on target system: ['grub-pc']
  configuring iscsi service
  configuring raid (mdadm) service
  configuring NVMe over TCP
  installing kernel
  setting up swap
  apply networking config
  writing etc/fstab
  configuring multipath
  updating packages on target system
  configuring pollinate user-agent on target
  updating initramfs configuration
  configuring target system bootloader
  installing grub to target devices
  copying metadata from /cdrom
final system configuration
  calculating extra packages to install
  configuring cloud-init
  downloading and installing security updates
  curtin command in-target
  restoring apt configuration
  curtin command in-target
subiquity/Late/run:
```

At the bottom of the terminal window, there are two links: '[View full log]' and '[Reiniciar ahora]', which is highlighted in green. The bottom of the window shows a taskbar with various icons and the text 'Ctrl Derecho'.

20º Cuando la máquina se reinicie, hay que iniciar sesión con el usuario y contraseña indicados en el paso “15º”:



The screenshot shows the 'SRV [Corriendo] - Oracle VirtualBox' window. The menu bar is the same as in the previous image. The main area is a terminal window displaying the following text:

```
Ubuntu 24.04.3 LTS rafa-teii tty1
rafa-teii login: rafa-teii
Password: _
```

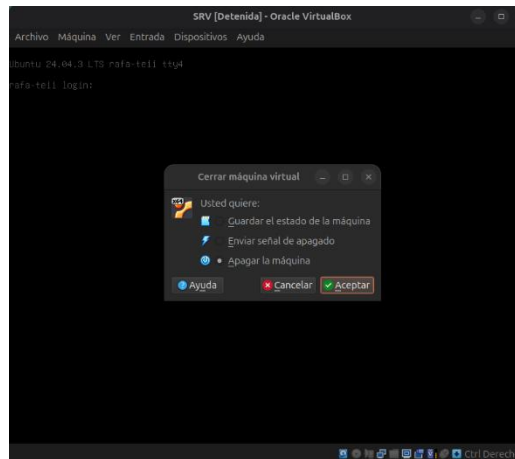
The bottom of the window shows a taskbar with various icons and the text 'Ctrl Derecho'.

21º Tras iniciar sesión, escribir las siguientes instrucciones: *sudo apt-get update* y después: *sudo apt install net-tools*.

22º Introducir la instrucción *ifconfig* y observar que en la salida, para la interfaz “enp0s8” aparece la dirección IP que se indicó durante la configuración de Ubuntu.

```
enp0s8: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
        inet 192.168.56.10 netmask 255.255.255.0 broadcast 192.168.56.255
```

23º Por último, cerrar y apagar la máquina virtual (también se podría hacer mediante el comando *VBoxManage controlvm SRV poweroff* en la terminal de la máquina principal):



```
(base) rafa@portatilrafa:~$ VBoxManage controlvm SRV poweroff
0%...10%...20%...30%...40%...50%...60%...70%...80%...90%...100%
```

3. Ejercicio 3. Operaciones en la VM SRV:

- Instalar nginx.
- Crear una página en `/var/www/html/index.html` que muestre el nombre SRV y una línea “VirtualBox OK”.
- Activar el servicio para que arranque al inicio.

Para resolver este ejercicio, primero hay que iniciar la máquina virtual SRV, para ello se usa el comando *VBoxManage startvm SRV*

```
(base) rafa@portatilrafa:~$ VBoxManage startvm SRV
Waiting for VM "SRV" to power on...
VM "SRV" has been successfully started.
```

Una vez iniciada, se instala nginx mediante la orden *sudo apt install nginx*

Tras ello, se crea la página en `/var/www/html/index.html` mediante: *touch /var/www/html/index.html* y después, mediante *sudo nano /var/www/html/index.html*, se introduce el contenido que se puede apreciar en la siguiente imagen:

```
GNU nano 7.2 /var/www/html/index.html
<html>
<head>
  <title>SRV</title>
</head>
<body bgcolor="lightgray">
  <h1>VirtualBox OK</h1>
</body>
</html>_
```

Ahora, se activa nginx para que arranque al inicio con: *sudo systemctl enable nginx*

```
rafa-teii@rafa-teii:~$ sudo systemctl enable nginx
Synchronizing state of nginx.service with SysV service script with /usr/lib/systemd/systemd-sysv-ins
tall.
Executing: /usr/lib/systemd/systemd-sysv-install enable nginx
```

4. Ejercicio 4. En SRV, configurar el port forwarding en el adaptador NAT:

- a. Host 8081 → guest 80.
- b. Host 2222 → guest 22.

En este ejercicio se añaden dos reglas de port forwarding en el adaptador 1, el cual se configuró en modo NAT, lo que permite que la máquina virtual salga a red a través del host, pero los servicios que se ejecutan dentro de la VM no son accesibles directamente desde el host salvo que se definan reglas de reenvío de puertos.

Para realizar esta configuración, se usará el subcomando *natpf1* que significa NAT Port Forwarding sobre el adaptador 1, y será necesario indicar el nombre de la regla, el protocolo, los puertos del host y los puertos dentro de la máquina virtual.

Además, hay que tener en cuenta que para poder realizar esta configuración la máquina virtual debe estar apagada.

Entonces, los comandos necesarios para esta configuración son:

```
VBoxManage modifyvm SRV --natpf1 "nginx,tcp,,8081,,80"
```

```
VBoxManage modifyvm SRV --natpf1 "ssh,tcp,,2222,,22"
```

```
(base) rafa@portatilrafa:~$ VBoxManage modifyvm SRV --natpf1 "nginx,tcp,,8081,,80"
(base) rafa@portatilrafa:~$ VBoxManage modifyvm SRV --natpf1 "ssh,tcp,,2222,,22"
```

De esta forma, al acceder al puerto 8081 del host, virtualbox redirigirá la conexión al puerto 80 de la máquina virtual, donde nginx está escuchando. De la misma forma, el puerto 2222 del host será redirigido al puerto 22 de la máquina virtual para permitir acceso al ssh.

5. Ejercicio 5. Crear una VM llamada CLI con:

- a. 1 GB RAM, 1 CPU, 15 GB disco SATA (VDI).
- b. Ubuntu Server 24.04.
- c. Adaptador 1: NAT.
- d. Adaptador 2: Host-Only (vboxnet0) con IP fija 192.168.56.20.
- e. Instalar curl.

En este ejercicio se crea una segunda máquina virtual llamada CLI, y se utilizarán los mismos comandos indicados en el ejercicio 2, pero cambiando algunos datos, por lo que la resolución será más directa.

Primero, se crea la máquina virtual y se registra con el comando *VBoxManage createvm --name CLI --ostype Ubuntu_64 --register*

```
(base) rafa@portatilrafa:~$ VBoxManage createvm --name CLI --ostype Ubuntu_64 --register
Virtual machine 'CLI' is created and registered.
UUID: e47a279a-f649-48a8-86f6-f7f507608966
Settings file: '/home/rafa/VirtualBox VMs/CLI/CLI.vbox'
```

Ahora, se configura la memoria, CPU y los adaptadores de red, el primero en modo NAT y el segundo en Host-Only conectado a vboxnet0, se utiliza el comando *VBoxManage modifyvm CLI --memory 1024 --cpus 1 --nic1 nat --nic2 hostonly --hostonlyadapter2 vboxnet0*

```
(base) rafa@portatilrafa:~$ VBoxManage modifyvm CLI --memory 1024 --cpus 1 --nic1 nat --nic2 hostonly --hostonlyadapter2 vboxnet0
```

Tras esto, se crea el disco virtual VDI de 15 GB, mediante el comando *VBoxManage createhd --filename /home/rafa/TEII/bloque2/disco_cli.vdi --size 15360*

```
(base) rafa@portatilrafa:~$ VBoxManage createhd --filename /home/rafa/TEII/bloque2/disco_cli.vdi --size 15360
0%...10%...20%...30%...40%...50%...60%...70%...80%...90%...100%
Medium created. UUID: 168c8731-5de7-4798-8149-1d6094ed7dda
```

Después, se añade al controlador SATA y se asocia el disco VDI a la máquina, mediante los comandos:

1º *VBoxManage storagectl CLI --name "SATA Controller" --add sata --controller IntelAhci*

2º *VBoxManage storageattach CLI --storagectl "SATA Controller" --type hdd --port 0 --device 0 --medium /home/rafa/TEII/bloque2/disco_cli.vdi*

```
(base) rafa@portatilrafa:~$ VBoxManage storagectl CLI --name "SATA Controller" --add sata --controller IntelAhci
(base) rafa@portatilrafa:~$ VBoxManage storageattach CLI --storagectl "SATA Controller" --type hdd --port 0 --device 0 --medium /home/rafa/TEII/bloque2/disco_cli.vdi
```

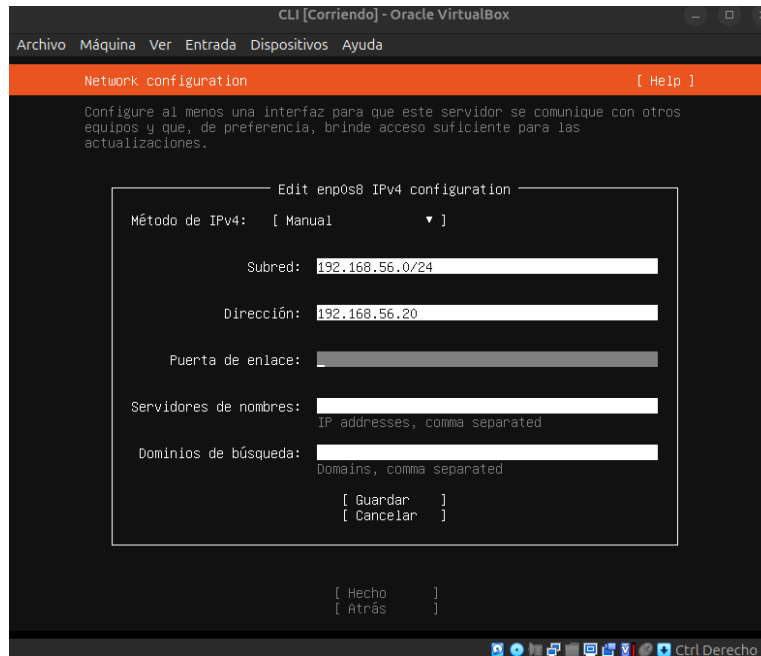
Ahora, se añade la unidad de DVD virtual y se monta la imagen ISO de Ubuntu Server con los comandos:

1º *VBoxManage storagectl CLI --name "IDE Controller" --add ide --controller PIIX4*

2º *VBoxManage storageattach CLI --storagectl "IDE Controller" --type DVDDrive --port 1 --device 0 --medium /home/rafa/TEII/bloque2/ubuntu-24.04.3-live-server-amd64.iso*

```
(base) rafa@portatilrafa:~$ VBoxManage storagectl CLI --name "IDE Controller" --add ide --controller PIIX4
(base) rafa@portatilrafa:~$ VBoxManage storageattach CLI --storagectl "IDE Controller" --type DVDDrive --port 1 --device 0 --medium /home/rafa/TEII/bloque2/ubuntu-24.04.3-live-server-amd64.iso
```

Tras esto, hay que lanzar la máquina virtual con el comando *VBoxManage startvm CLI* y realizar la instalación del sistema operativo. Se seguirán los mismos pasos que para la máquina virtual SRV (ejercicio 2) pero esta vez indicando la dirección IP 192.168.56.20, como se puede ver en la siguiente imagen:



Tras terminar toda la configuración e instalación del sistema operativo, la máquina se reiniciará. Entonces, se tendrá que iniciar sesión para instalar curl. Primero, se introducirá el comando *sudo apt-get update* y después para instalar curl se utilizará el comando *sudo apt install curl*.

```
rafa-teii-cli@rafa-teii-cli:~$ sudo apt-get update
[sudo] password for rafa-teii-cli:
Obj:1 http://es.archive.ubuntu.com/ubuntu noble InRelease
Obj:2 http://es.archive.ubuntu.com/ubuntu noble-updates InRelease
Obj:3 http://es.archive.ubuntu.com/ubuntu noble-backports InRelease
Obj:4 http://security.ubuntu.com/ubuntu noble-security InRelease
Des:5 http://es.archive.ubuntu.com/ubuntu noble/main Translation-es [325 kB]
Des:6 http://es.archive.ubuntu.com/ubuntu noble/restricted Translation-es [816 B]
Des:7 http://es.archive.ubuntu.com/ubuntu noble/universe Translation-es [1.371 kB]
Des:8 http://es.archive.ubuntu.com/ubuntu noble/multiverse Translation-es [63,1 kB]
Descargados 1.759 kB en 1s (1.900 kB/s)
Leyendo lista de paquetes... Hecho
rafa-teii-cli@rafa-teii-cli:~$ sudo apt install curl
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
curl ya está en su versión más reciente (8.5.0-2ubuntu10.8).
fijado curl como instalado manualmente.
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 60 no actualizados.
```

NOTA: he tenido un problema para instalar y configurar Ubuntu en el que la instalación daba error debido a la cantidad de RAM asignada. Por tanto, para poder realizar la instalación, antes de volver a iniciar la máquina virtual he usado el comando *VBoxManage modifyvm CLI --memory 2048* y, tras completar la instalación, he vuelto a apagar a la máquina para cambiar de nuevo la RAM asignada a la pedida por el ejercicio

con *VBoxManage modifyvm CLI --memory 1024*. Después, he vuelto a encender la máquina virtual para instalar *curl* tal y como indiqué antes.

```
(base) rafa@portatilrafa:~/TEII/bloque2$ VBoxManage modifyvm CLI --memory 2048
(base) rafa@portatilrafa:~/TEII/bloque2$ VBoxManage startvm CLI
Waiting for VM "CLI" to power on...
VM "CLI" has been successfully started.
(base) rafa@portatilrafa:~/TEII/bloque2$ VBoxManage controlvm CLI poweroff
0%...10%...20%...30%...40%...50%...60%...70%...80%...90%...100%
(base) rafa@portatilrafa:~/TEII/bloque2$ VBoxManage modifyvm CLI --memory 1024
```

6. Ejercicio 6. Modificar los recursos de CLI:

a. 2 GB RAM, 2 CPU, limite CPU 60%.

En este ejercicio se modifica la configuración de la máquina virtual creada en el ejercicio anterior, aumentando la memoria RAM a 2GB, el número de procesadores a 2 y estableciendo un límite de uso de CPU al 60%. Para realizar esta modificación, se utiliza el comando *VBoxManage modifyvm* con las opciones *--memory 2048* para asignar 2GB de RAM, la opción *--cpus 2* para establecer el número de procesadores a 2 y, por último, la opción *--cpuexecutioncap 60*, que fija el límite de uso de la CPU al 60%. El comando final quedaría: *VBoxManage modifyvm CLI --memory 2048 --cpus 2 --cpuexecutioncap 60*

```
(base) rafa@portatilrafa:~/TEII/bloque2$ VBoxManage modifyvm CLI --memory 2048 --cpus 2 --cpuexecutioncap 60
```

Finalmente, mediante el comando *VBoxManage showvminfo CLI*, se puede comprobar que la máquina ha quedado configurada correctamente:

```
(base) rafa@portatilrafa:~$ VBoxManage showvminfo CLI
Name: CLI
Encryption: disabled
Groups: /
Platform Architecture: x86
Guest OS: Ubuntu (64-bit)
UUID: f5889d7d-22e7-4ccc-8e01-f0e8c814636e
Config file: /home/rafa/VirtualBox VMs/CLI/CLI.vbox
Snapshot folder: /home/rafa/VirtualBox VMs/CLI/Snapshots
Log folder: /home/rafa/VirtualBox VMs/CLI/Logs
Hardware UUID: f5889d7d-22e7-4ccc-8e01-f0e8c814636e
Memory size: 2048MB
Page Fusion: disabled
VRAM size: 8MB
CPU exec cap: 60%
CPUProfile: host
Chipset: piix3
Firmware: BIOS
Number of CPUs: 2
```

7. Ejercicio 7. Crear un linked clone del servidor llamado SRV-CLONE:

- Cambiar hostname .
- Configurar IP fija en host-only: 192.168.56.30.

- c. Mantener nginx mostrando una página que dice: “CLONE OK”.
- d. Mostrar la salida de curl a las dos direcciones de SRV y SRV-CLONE desde CLI.

Un linked clone es un clon que crea discos diferenciales basados en los discos de la máquina origen. Por eso, se apoya en una snapshot del sistema original y depende de ese punto.

En este ejercicio, se toma una snapshot de la máquina SRV, y a partir de ella, se genera la nueva máquina SRV-CLONE. Después, hay que realizar una modificación en la configuración de Ubuntu de la nueva máquina para cambiar su hostname y su IP fija.

Además, como esta máquina se crea a partir del estado de SRV, nginx ya está instalado. Por tanto, simplemente hay que modificar el contenido de `/var/www/html/index.html` para que la máquina sirva la página que muestre “CLONE OK”. Después, desde la máquina CLI, el uso de `curl` sobre las direcciones IP 192.168.56.10 (máquina SRV) y 192.168.56.30 (SRV-CLONE) permite demostrar que ambas máquinas están accesibles por la red Host-Only y que cada una responde con su contenido correspondiente.

Por tanto, para realizar este ejercicio, primero se crea una snapshot de la máquina SRV mediante el comando `VBoxManage snapshot SRV take "snapshot-srv-clone"` y ahora se genera el linked clone a partir de esta snapshot mediante el comando `VBoxManage clonevm SRV --name "SRV-CLONE" --snapshot "snapshot-srv-clone" --options link --register`

```
(base) rafa@portatilrafa:~/TEII/bloque2$ VBoxManage snapshot SRV take "snapshot-srv-clone"
0%...10%...20%...30%...40%...50%...60%...70%...80%...90%...100%
Snapshot taken. UUID: dd6936c4-1623-45c6-8538-a2174b843952
(base) rafa@portatilrafa:~/TEII/bloque2$ VBoxManage clonevm SRV --name "SRV-CLONE" --snapshot "snapshot-srv-clone" --options link --register
0%...10%...20%...30%...40%...50%...60%...70%...80%...90%...100%
Machine has been successfully cloned as "SRV-CLONE"
```

Ahora se inicia la máquina con el comando `VBoxManage startvm SRV-CLONE`:

```
(base) rafa@portatilrafa:~/TEII/bloque2$ VBoxManage startvm SRV-CLONE
Waiting for VM "SRV-CLONE" to power on...
VM "SRV-CLONE" has been successfully started.
```

Una vez iniciada la máquina, se cambia el hostname introduciendo el comando `sudo hostnamectl set-hostname SRV-CLONE`

```
rafa-teii@rafa-teii:~$ sudo hostnamectl set-hostname SRV-CLONE
```

Para poder cambiar la IP fija, hay que modificar el fichero `/etc/netplan/50-cloud-init.yaml` y, en la interfaz `enp0s8` cambiar la IP 192.168.56.10 por 192.168.56.30. Para ello se utilizará el comando `sudo nano /etc/netplan/50-cloud-init.yaml` y se aplicarán los cambios mediante el comando `sudo netplan apply`, el fichero debe quedar así:

```
GNU nano 7.2 /etc/netplan/50-cloud-init.yaml
network:
  version: 2
  ethernet:
    enp0s8:
      dhcp4: true
    enp0s8:
      addresses:
        - "192.168.56.30/24"
```

Ahora, se modifica el fichero `index.html` para que muestre “CLONE OK”, para ello se utiliza la orden `sudo nano /var/www/html/index.html` y se modifica su contenido para que quede como a continuación:

```
GNU nano 7.2 /var/www/html/index.html
<html>
<head>
    <title>SRV-CLONE</title>
</head>
<body bgcolor="lightgray">
    <h1>CLONE OK</h1>
</body>
</html>
```

Ahora, se inician las máquinas SRV y CLI (SRV-CLONE ya está encendida) mediante los comandos `VBoxManage startvm SRV` y `VBoxManage startvm CLI`, se inicia sesión en ambas y tras ello se introducen en la máquina CLI las ordenes `curl -s http://192.168.56.10` y después `curl -s http://192.168.56.30`

```
(base) rafa@portatilrafa:~/TEII/bloque2$ VBoxManage startvm SRV-CLONE
Waiting for VM "SRV-CLONE" to power on...
VM "SRV-CLONE" has been successfully started.
(base) rafa@portatilrafa:~/TEII/bloque2$ VBoxManage startvm CLI
Waiting for VM "CLI" to power on...
VM "CLI" has been successfully started.
(base) rafa@portatilrafa:~/TEII/bloque2$ VBoxManage startvm SRV
Waiting for VM "SRV" to power on...
VM "SRV" has been successfully started.
```

Y en la máquina virtual CLI:

```
CLI [Corriendo] - Oracle VirtualBox
Archivo Máquina Ver Entrada Dispositivos Ayuda
rafa-teii-cli@rafa-teii-cli:~$ curl -s http://192.168.56.10
<html>
<head>
    <title>SRV</title>
</head>
<body bgcolor="lightgray">
    <h1>VirtualBox OK</h1>
</body>
</html>
rafa-teii-cli@rafa-teii-cli:~$ curl -s http://192.168.56.30
<html>
<head>
    <title>SRV-CLONE</title>
</head>
<body bgcolor="lightgray">
    <h1>CLONE OK</h1>
</body>
</html>
```

Como se puede observar, las peticiones devuelven las páginas web de SRV y SRV-CLONE respectivamente.

Tras esto, se apagan las tres máquinas virtuales:

```
(base) rafa@portatilrafa:~/TEII/bloque2$ VBoxManage controlvm SRV-CLONE poweroff
0%...10%...20%...30%...40%...50%...60%...70%...80%...90%...100%
(base) rafa@portatilrafa:~/TEII/bloque2$ VBoxManage controlvm SRV poweroff
0%...10%...20%...30%...40%...50%...60%...70%...80%...90%...100%
(base) rafa@portatilrafa:~/TEII/bloque2$ VBoxManage controlvm CLI poweroff
0%...10%...20%...30%...40%...50%...60%...70%...80%...90%...100%
```

- **KVM/LIBVIRT/VIRSH**

1. **Ejercicio 1. Crear una máquina virtual KVM-SRV con Ubuntu Server 24.04, con las siguientes características:**

- a. La interfaz de red debe estar configurada en modo NAT de libvirt.
- b. 1 GB de RAM, 1 CPU, 12 GB de disco duro qcow.

Para resolver este ejercicio, primero hay que comprobar que la red NAT por defecto de libvirt está disponible. Esto se hace mediante el comando *virsh net-list*.

```
(base) rafa@portatilrafa:~$ virsh net-list
Name      State    Autostart  Persistent
-----
default   active   yes        yes
```

Como se puede ver en la salida anterior sí está disponible. Esa es la red NAT de libvirt que se utilizará. Ahora, se crea el disco virtual en formato qcow de 12 GB. Para ello, se utilizará el comando *sudo qemu-img create -f qcow2 /var/lib/libvirt/images/teii/KVM-SRV.qcow2 12G*

```
rafa@portatilrafa:~$ sudo qemu-img create -f qcow2 /var/lib/libvirt/images/teii/KVM-SRV.qcow2 12G
Formatting '/var/lib/libvirt/images/teii/KVM-SRV.qcow2', fmt=qcow2 cluster_size=65536 extended_l2=off
compression_type=zlib size=12884901888 lazy_refcounts=off refcount_bits=16
```

Ahora, se crea la máquina virtual con el comando *virt-install --virt-type kvm --name KVM-SRV --ram 1024 --vcpus=1 --disk path=/var/lib/libvirt/images/teii/KVM-SRV.qcow2 --network bridge=virbr0 --graphics vnc --os-variant=ubuntu24.04 --cdrom /var/lib/libvirt/images/teii/ubuntu-24.04.3-live-server-amd64.iso*. Tras introducir este comando, se crea la máquina virtual y se inicia la instalación de Ubuntu:

```
rafa@portatilrafa:~$ sudo virt-install --virt-type kvm --name KVM-SRV --ram 1024 --vcpus=1 --disk path=/var/lib/libvirt/images/teii/KVM-SRV.qcow2 --network bridge=virbr0 --graphics vnc --os-variant=ubuntu24.04 --cdrom /var/lib/libvirt/images/teii/ubuntu-24.04.3-live-server-amd64.iso
WARNING Requested memory 1024 MiB is less than the recommended 3072 MiB for OS ubuntu24.04

Empezando la instalación...
Creando dominio... | 00:00:00
Running graphical console command: virt-viewer --connect qemu:///system --wait KVM-SRV
```

Tras esto, se ha realizado la instalación y configuración de Ubuntu (igual que como se explicaba en el ejercicio 2 de la parte anterior)

2. **Ejercicio 2. Dentro de la VM, instalar el servicio apache2 y arrancarlo.**

Primero, se inicia la máquina virtual mediante el comando *sudo virsh start KVM-SRV* y después se introduce *virt-viewer KVM-SRV* para conectarse a la máquina:

```
rafa@portatilrafa:~$ sudo virsh start KVM-SRV
Domain 'KVM-SRV' started

rafa@portatilrafa:~$ virt-viewer KVM-SRV
```

Ahora, tras iniciar sesión, para instalar el servicio apache2 se utiliza la orden *sudo apt install apache2*, tal y como se ve en la siguiente imagen:

```
rafa-kvm-srv@rafa-kvm-srv:~$ sudo apt install apache2
[sudo] password for rafa-kvm-srv:
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
apache2-bin apache2-data apache2-utils libapr1t64 libaprutil1-dbd-sqlite3 libaprutil1-ldap libaprutil1t64 liblua5.4-0 ssl-cert
Paquetes sugeridos:
apache2-doc apache2-suexec-pristine | apache2-suexec-custom www-browser
Se instalarán los siguientes paquetes NUEVOS:
apache2 apache2-bin apache2-data apache2-utils libapr1t64 libaprutil1-dbd-sqlite3 libaprutil1-ldap libaprutil1t64 liblua5.4-0 ssl-cert
0 actualizados, 10 nuevos se instalarán, 0 para eliminar y 49 no actualizados.
Se necesita descargar 2,091 kB de archivos.
Se utilizarán 8,113 kB de espacio de disco adicional después de esta operación.
¿Desea continuar? [S/n] S
```

Para dejar el servicio instalado se ejecuta `sudo systemctl restart apache2.service`

```
rafa-kvm-srv@rafa-kvm-srv:~$ sudo systemctl restart apache2.service
```

3. Ejercicio 3. Desde el host, obtener IP y comprobar el acceso con curl.

Para obtener la IP de la máquina, se ejecutará `sudo virsh net-dhcp-leases default` para observar la dirección que le ha concedido DHCP.

```
rafa@portatilrafa:~$ sudo virsh net-dhcp-leases default
Expiry Time      MAC address      Protocol  IP address      Hostname      Client ID or DUID
-----
2026-03-26 22:31:12  52:54:00:a1:f7:41  ipv4       192.168.122.4/24  rafa-kvm-srv  ff:56:50:4d:98:00:02:00:00:ab:11:93:6b:cc:94:2b:12:02:dd
```

Como se puede observar, se le ha asignado la dirección 192.168.122.4.

Ahora, se comprueba el acceso con curl ejecutando desde la máquina principal `curl http://192.168.122.4`

```
rafa@portatilrafa:~$ curl http://192.168.122.4
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <!--
    Modified from the Debian original for Ubuntu
    Last updated: 2022-03-22
    See: https://launchpad.net/bugs/1966004
  -->
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
    <title>Apache2 Ubuntu Default Page: It works</title>
    <style type="text/css" media="screen">
```

Con la imagen anterior, se puede comprobar que apache está respondiendo.

4. Ejercicio 4. Obtener el fichero de configuración de la máquina en formato XML, y añadirlo/pegarlo a la documentación tal cual se genera.

Para obtener el fichero de configuración en formato XML, teniendo la máquina virtual apagada (`virsh shutdown KVM-SRV`), hay que ejecutar en la máquina principal `sudo virsh dumpxml KVM-SRV > KVM-SRV.xml`

```
rafa@portatilrafa:~$ virsh shutdown KVM-SRV
Domain 'KVM-SRV' is being shutdown

rafa@portatilrafa:~$ sudo virsh dumpxml KVM-SRV > KVM-SRV.xml
```

El contenido de KVM-SRV.xml es:

```
<domain type='kvm'>
  <name>KVM-SRV</name>
```

```
<uuid>be8387ea-a3f4-4a07-baba-d82a3f6240e3</uuid>

<metadata>

  <libosinfo:libosinfo xmlns:libosinfo="http://libosinfo.org/xmlns/libvirt/domain/1.0">

    <libosinfo:os id="http://ubuntu.com/ubuntu/24.04"/>

  </libosinfo:libosinfo>

</metadata>

<memory unit='KiB'>1048576</memory>

<currentMemory unit='KiB'>1048576</currentMemory>

<vcpu placement='static'>1</vcpu>

<resource>

  <partition>/machine</partition>

</resource>

<os>

  <type arch='x86_64' machine='pc-q35-9.2'>hvm</type>

  <boot dev='hd'>

  <boot dev='cdrom'>

</os>

<features>

  <acpi/>

  <apic/>

</features>

<cpu mode='host-passthrough' check='none' migratable='on'>

<clock offset='utc'>

  <timer name='rtc' tickpolicy='catchup'>

  <timer name='pit' tickpolicy='delay'>

  <timer name='hpet' present='no'>

</clock>

<on_poweroff>destroy</on_poweroff>

<on_reboot>destroy</on_reboot>
```

```
<on_crash>destroy</on_crash>

<pm>

  <suspend-to-mem enabled='no'/>

  <suspend-to-disk enabled='no'/>

</pm>

<devices>

  <emulator>/usr/bin/qemu-system-x86_64</emulator>

  <disk type='file' device='disk'>

    <driver name='qemu' type='qcow2'/>

    <source file='/var/lib/libvirt/images/teii/KVM-SRV.qcow2'/>

    <backingStore/>

    <target dev='vda' bus='virtio'/>

    <address type='pci' domain='0x0000' bus='0x04' slot='0x00' function='0x0'/>

  </disk>

  <disk type='file' device='cdrom'>

    <driver name='qemu' type='raw'/>

    <source file='/var/lib/libvirt/images/teii/ubuntu-24.04.3-live-server-amd64.iso'/>

    <backingStore/>

    <target dev='sda' bus='sata'/>

    <readonly/>

    <address type='drive' controller='0' bus='0' target='0' unit='0'/>

  </disk>

  <controller type='usb' index='0' model='qemu-xhci' ports='15'>

    <address type='pci' domain='0x0000' bus='0x02' slot='0x00' function='0x0'/>

  </controller>

  <controller type='pci' index='0' model='pcie-root'/>

  <controller type='pci' index='1' model='pcie-root-port'>

    <model name='pcie-root-port'/>

    <target chassis='1' port='0x10'/>

  </controller>

</devices>
```

```
<address type='pci' domain='0x0000' bus='0x00' slot='0x02' function='0x0'
multifunction='on'/>
```

```
</controller>
```

```
<controller type='pci' index='2' model='pcie-root-port'>
```

```
<model name='pcie-root-port'/>
```

```
<target chassis='2' port='0x11'/>
```

```
<address type='pci' domain='0x0000' bus='0x00' slot='0x02' function='0x1'/>
```

```
</controller>
```

```
<controller type='pci' index='3' model='pcie-root-port'>
```

```
<model name='pcie-root-port'/>
```

```
<target chassis='3' port='0x12'/>
```

```
<address type='pci' domain='0x0000' bus='0x00' slot='0x02' function='0x2'/>
```

```
</controller>
```

```
<controller type='pci' index='4' model='pcie-root-port'>
```

```
<model name='pcie-root-port'/>
```

```
<target chassis='4' port='0x13'/>
```

```
<address type='pci' domain='0x0000' bus='0x00' slot='0x02' function='0x3'/>
```

```
</controller>
```

```
<controller type='pci' index='5' model='pcie-root-port'>
```

```
<model name='pcie-root-port'/>
```

```
<target chassis='5' port='0x14'/>
```

```
<address type='pci' domain='0x0000' bus='0x00' slot='0x02' function='0x4'/>
```

```
</controller>
```

```
<controller type='pci' index='6' model='pcie-root-port'>
```

```
<model name='pcie-root-port'/>
```

```
<target chassis='6' port='0x15'/>
```

```
<address type='pci' domain='0x0000' bus='0x00' slot='0x02' function='0x5'/>
```

```
</controller>
```

```
<controller type='pci' index='7' model='pcie-root-port'>
```

```
<model name='pcie-root-port'/>
<target chassis='7' port='0x16'/>
<address type='pci' domain='0x0000' bus='0x00' slot='0x02' function='0x6'/>
</controller>
<controller type='pci' index='8' model='pcie-root-port'>
  <model name='pcie-root-port'/>
  <target chassis='8' port='0x17'/>
  <address type='pci' domain='0x0000' bus='0x00' slot='0x02' function='0x7'/>
</controller>
<controller type='pci' index='9' model='pcie-root-port'>
  <model name='pcie-root-port'/>
  <target chassis='9' port='0x18'/>
  <address type='pci' domain='0x0000' bus='0x00' slot='0x03' function='0x0'
multifunction='on'/>
</controller>
<controller type='pci' index='10' model='pcie-root-port'>
  <model name='pcie-root-port'/>
  <target chassis='10' port='0x19'/>
  <address type='pci' domain='0x0000' bus='0x00' slot='0x03' function='0x1'/>
</controller>
<controller type='pci' index='11' model='pcie-root-port'>
  <model name='pcie-root-port'/>
  <target chassis='11' port='0x1a'/>
  <address type='pci' domain='0x0000' bus='0x00' slot='0x03' function='0x2'/>
</controller>
<controller type='pci' index='12' model='pcie-root-port'>
  <model name='pcie-root-port'/>
  <target chassis='12' port='0x1b'/>
  <address type='pci' domain='0x0000' bus='0x00' slot='0x03' function='0x3'/>
```

</controller>

<controller type='pci' index='13' model='pcie-root-port'>

<model name='pcie-root-port'/>

<target chassis='13' port='0x1c'/>

<address type='pci' domain='0x0000' bus='0x00' slot='0x03' function='0x4'/>

</controller>

<controller type='pci' index='14' model='pcie-root-port'>

<model name='pcie-root-port'/>

<target chassis='14' port='0x1d'/>

<address type='pci' domain='0x0000' bus='0x00' slot='0x03' function='0x5'/>

</controller>

<controller type='sata' index='0'>

<address type='pci' domain='0x0000' bus='0x00' slot='0x1f' function='0x2'/>

</controller>

<controller type='virtio-serial' index='0'>

<address type='pci' domain='0x0000' bus='0x03' slot='0x00' function='0x0'/>

</controller>

<interface type='bridge'>

<mac address='52:54:00:a1:f7:41'/>

<source bridge='virbr0'/>

<model type='virtio'/>

<address type='pci' domain='0x0000' bus='0x01' slot='0x00' function='0x0'/>

</interface>

<serial type='pty'>

<target type='isa-serial' port='0'>

<model name='isa-serial'/>

</target>

</serial>

<console type='pty'>

```
<target type='serial' port='0'/>
</console>
<channel type='unix'>
  <target type='virtio' name='org.qemu.guest_agent.0'/>
  <address type='virtio-serial' controller='0' bus='0' port='1'/>
</channel>
<input type='tablet' bus='usb'>
  <address type='usb' bus='0' port='1'/>
</input>
<input type='mouse' bus='ps2'/>
<input type='keyboard' bus='ps2'/>
<graphics type='vnc' port='-1' autoport='yes' listen='127.0.0.1'>
  <listen type='address' address='127.0.0.1'/>
</graphics>
<audio id='1' type='none'/>
<video>
  <model type='virtio' heads='1' primary='yes'/>
  <address type='pci' domain='0x0000' bus='0x00' slot='0x01' function='0x0'/>
</video>
<watchdog model='itco' action='reset'/>
<memballoon model='virtio'>
  <address type='pci' domain='0x0000' bus='0x05' slot='0x00' function='0x0'/>
</memballoon>
<rng model='virtio'>
  <backend model='random'>/dev/urandom</backend>
  <address type='pci' domain='0x0000' bus='0x06' slot='0x00' function='0x0'/>
</rng>
</devices>
<seclabel type='dynamic' model='apparmor' relabel='yes'/>
```

```
<seclabel type='dynamic' model='dac' relabel='yes'/>
```

```
</domain>
```

5. Ejercicio 5. Apagar la máquina y destruirla.

Para apagar la máquina se utiliza el comando *sudo virsh shutdown KVM-SRV* y para destruirla se utiliza *sudo virsh undefine KVM-SRV*, como se puede observar en la siguiente imagen:

```
rafa@portatilrafa:~$ sudo virsh shutdown KVM-SRV
Domain 'KVM-SRV' is being shutdown

rafa@portatilrafa:~$ sudo virsh undefine KVM-SRV
Domain 'KVM-SRV' has been undefined
```

Como se puede observar a continuación, se ha destruido correctamente:

```
rafa@portatilrafa:~$ virsh list --all
 Id   Name      State
-----
rafa@portatilrafa:~$
```

- **Vagrant**

1. **Ejercicio 1. Crear una Box de Vagrant con:**

- a. **Una box Linux (bionic64 o equivalente)**
- b. **Usar el provisioning para instalar apache2 y servir un index.html desde la carpeta del proyecto (synced folder).**

Para realizar este ejercicio, primero se crea un directorio para el proyecto de vagrant con `mkdir teii-vagrant`, y ahora, dentro de este directorio, se inicializa vagrant con la box hashicorp/bionic64 mediante `vagrant init hashicorp/bionic64`

```
(base) rafa@portatilrafa:~$ mkdir teii-vagrant
(base) rafa@portatilrafa:~$ cd teii-vagrant/
(base) rafa@portatilrafa:~/teii-vagrant$ vagrant init hashicorp/bionic64
A 'Vagrantfile' has been placed in this directory. You are now
ready to 'vagrant up' your first virtual environment! Please read
the comments in the Vagrantfile as well as documentation on
'vagrantup.com' for more information on using Vagrant.
```

Ahora, se crea la carpeta html y el fichero index.html, que será el contenido servido desde la carpeta compartida. El provisioner enlazará /var/www con /vagrant, por lo que el

```
GNU nano 8.3 index.html
<html>
  <body>
    <h1>Proyecto de Vagrant</h1>
  </body>
</html>
```

fichero acabará siendo accesible desde Apache a través de la carpeta sincronizada. Para esto se hace primero `mkdir html` después `cd html/`, tras esto, `touch index.html` y por último `nano index.html` y se le introduce el siguiente contenido:

Las órdenes utilizadas son:

```
(base) rafa@portatilrafa:~/teii-vagrant$ mkdir html
(base) rafa@portatilrafa:~/teii-vagrant$ cd html/
(base) rafa@portatilrafa:~/teii-vagrant/html$ touch index.html
(base) rafa@portatilrafa:~/teii-vagrant/html$ nano index.html
```

Tras esto, se crea en el directorio del proyecto el scrip de provisioning `bootstrap.sh`, cuyo contenido es el siguiente:

```
GNU nano 8.3 bootstrap.sh
#!/usr/bin/env bash
apt-get update -y
apt-get install -y apache2
if ! [ -L /var/www ]; then
  rm -rf /var/www
  ln -fs /vagrant /var/www
fi
```

Ahora, se modifica el fichero Vagrantfile para que use la box y ejecute el provisioner al arrancar la máquina virtual. El contenido del Vagrantfile es:

```
rafa@portatilrafa: ~/teii-vagrant
GNU nano 8.3 Vagrantfile
# -*- mode: ruby -*-
# vi: set ft=ruby :

# All Vagrant configuration is done below. The "2" in Vagrant.configure
# configures the configuration version (we support older styles for
# backwards compatibility). Please don't change it unless you know what
# you're doing.
Vagrant.configure("2") do |config|
  # The most common configuration options are documented and commented below.
  # For a complete reference, please see the online documentation at
  # https://docs.vagrantup.com.

  # Every Vagrant development environment requires a box. You can search for
  # boxes at https://vagrantcloud.com/search.
  config.vm.box = "hashicorp/bionic64"
  config.vm.provision :shell, path: "bootstrap.sh"
  # Disable automatic box update checking. If you disable this, then
  # boxes will only be checked for updates when the user runs
```

(el resto del fichero no se ha tocado)

Por último, se inicia la máquina virtual con *vagrant up*

```
(base) rafa@portatilrafa:~/teii-vagrant$ vagrant up
Bringing machine 'default' up with 'virtualbox' provider...
==> default: Importing base box 'hashicorp/bionic64'...
==> default: Matching MAC address for NAT networking...
==> default: Checking if box 'hashicorp/bionic64' version '1.0.282' is up to date...
==> default: Setting the name of the VM: teii-vagrant_default_1774571314720_21361
==> default: Fixed port collision for 22 => 2222. Now on port 2200.
==> default: Clearing any previously set network interfaces...
==> default: Preparing network interfaces based on configuration...
```

Tras esto, desde dentro de la máquina virtual, ya se puede acceder a la página de inicio del servidor web. Para ello, se utiliza *vagrant ssh* para entrar a la máquina y, una vez dentro, se utiliza *wget -qO- 127.0.0.1* para acceder a dicha página:

```
(base) rafa@portatilrafa:~/teii-vagrant$ vagrant ssh
Welcome to Ubuntu 18.04.3 LTS (GNU/Linux 4.15.0-58-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/advantage

System information as of Fri Mar 27 00:29:57 UTC 2026

System load:  0.57               Processes:    94
Usage of /:   2.7% of 61.80GB    Users logged in: 0
Memory usage: 13%               IP address for eth0: 10.0.2.15
Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

https://ubuntu.com/engage/secure-kubernetes-at-the-edge

302 packages can be updated.
242 updates are security updates.

vagrant@vagrant:~$ wget -qO- 127.0.0.1
<html>
  <body>
    <h1>Proyecto de Vagrant</h1>
  </body>
</html>
```

2. Ejercicio 2. Configurar la VM de Vagrant para que:

- Exponga el servicio por port forwarding: host 8090 → guest 80.
- Se conecta a la misma red Host-Only vboxnet0 creada anteriormente con IP fija 192.168.56.50.
- Se configure con 1 GB de RAM y 1 CPU.

Para realizar estas configuraciones, hay que editar el Vagrantfile, añadiéndole el port forwarding, la red privada Host-Only con la IP fija 192.168.56.50 y las configuraciones de RAM y CPU. El contenido del fichero quedará así:

```
rafa@portatilrafa: ~/tei-vagrant
GNU nano 8.3 Vagrantfile
# -*- mode: ruby -*-
# vi: set ft=ruby :

# All Vagrant configuration is done below. The "2" in Vagrant.configure
# configures the configuration version (we support older styles for
# backwards compatibility). Please don't change it unless you know what
# you're doing.
Vagrant.configure("2") do |config|
  # The most common configuration options are documented and commented below.
  # For a complete reference, please see the online documentation at
  # https://docs.vagrantup.com.

  # Every Vagrant development environment requires a box. You can search for
  # boxes at https://vagrantcloud.com/search.
  config.vm.box = "hashicorp/bionic64"
  config.vm.provision :shell, path: "bootstrap.sh"
  config.vm.network :forwarded_port, guest: 80, host: 8090
  config.vm.network "private_network", ip: "192.168.56.50"
  config.vm.provider "virtualbox" do |v|
    v.memory = 1024
    v.cpus = 1
  end
end
```

Ahora, se aplican los cambios con *vagrant reload --provision*.

3. Ejercicio 3. Comprobar que:

- Desde el host: `curl http://127.0.0.1:8090`
- Desde CLI (192.168.56.20): `curl http://192.168.56.50`
- Desde la VM Vagrant: `curl http://192.168.56.10`

Desde la máquina principal se hace `curl http://127.0.0.1:8090`

```
(base) rafa@portatilrafa:~/tei-vagrant$ curl http://127.0.0.1:8090
<html>
  <body>
    <h1>Proyecto de Vagrant</h1>
  </body>
</html>
```

Así, se verifica el port forwarding configurado. Como en el vagrantfile se ha redirigido el puerto 8090 del host al 80 de la máquina virtual, al hacer curl 127.0.0.1:8090 desde el host se obtiene la página servida por Apache dentro de la máquina vagrant (el fichero index.html creado en el ejercicio 1 de este apartado).

Ahora, se inicia la máquina CLI con *VBoxManage startvm CLI* y en ella se ejecuta *curl http://192.168.56.50*

```
rafa-teii-cli@rafa-teii-cli:~$ curl http://192.168.56.50
<html>
  <body>
    <h1>Proyecto de Vagrant</h1>
  </body>
</html>
```

Esta comprobación se realiza desde la máquina virtual CLI, que está configurada con la IP 192.168.56.20 en la red 192.168.56.0/24. Como la máquina virtual de vagrant también se ha conectado a esa misma red Host-Only con la IP fija 192.168.56.50, CLI puede llegar directamente al servidor web de la máquina de vagrant por esa dirección privada. Por ello, se ve en la salida de curl la página web servida por Apache en la máquina de vagrant.

Por último, se inicia la máquina virtual SRV con *VBoxManage startvm SRV* (es la que está configurada con la dirección IP 192.168.56.10). Después se accede a la máquina virtual de Vagrant (ya estaba iniciada) con *vagrant ssh* y una vez dentro se ejecuta *curl http://192.168.56.10*

```
vagrant@vagrant:~$ curl http://192.168.56.10
<html>
<head>
  <title>SRV</title>
</head>
<body bgcolor="lightgray">
  <h1>VirtualBox OK</h1>
</body>
</html>
```

Ahora, como se ejecuta curl desde la máquina virtual de vagrant para comprobar que se puede acceder a SRV, lo que se ve es la página de SRV. Esta máquina fue configurada con la IP Host-Only 192.168.56.10 y con nginx sirviendo la página que se muestra en la imagen anterior.

4. Ejercicio 4. Limpieza de todas las VMs creadas en los apartados anteriores.

Para limpiar todas las máquinas virtuales creadas, primero hay que apagarlas:

- *VBoxManage controlvm SRV poweroff*
- *VBoxManage controlvm CLI poweroff*
- *VBoxManage controlvm SRV-CLONE poweroff*.

```
(base) rafa@portatilrafa:~/teii-vagrant$ VBoxManage controlvm SRV poweroff
0%...10%...20%...30%...40%...50%...60%...70%...80%...90%...100%
(base) rafa@portatilrafa:~/teii-vagrant$ VBoxManage controlvm CLI poweroff
0%...10%...20%...30%...40%...50%...60%...70%...80%...90%...100%
(base) rafa@portatilrafa:~/teii-vagrant$ VBoxManage controlvm SRV-CLONE poweroff
0%...10%...20%...30%...40%...50%...60%...70%...80%...90%...100%
```

Ahora, se eliminan:

- *VBoxManage unregistervm SRV --delete*
- *VBoxManage unregistervm CLI --delete*

- *VBoxManage unregistervm SRV-CLONE --delete*

La máquina KVM-SRV ya fue eliminada, se puede comprobar con *sudo virsh list --all* observando que no aparece:

```
(base) rafa@portatilrafa:~/teii-vagrant$ sudo virsh list --all
Id    Name    State
-----
```

Por último, para eliminar máquina virtual de Vagrant, hay que ejecutar *vagrant destroy* y confirmar con “y” desde el directorio del proyecto:

```
(base) rafa@portatilrafa:~/teii-vagrant$ vagrant destroy
default: Are you sure you want to destroy the 'default' VM? [y/N] y
==> default: Forcing shutdown of VM...
==> default: Destroying VM and associated drives...
```

2. Parte 2 – Contenedores con Docker

- Primer contenedor

Para este primer contenedor, se ha creado el directorio `contenedor1` con `mkdir contenedor1`, y dentro de este directorio se ha creado el script `mostrar-fecha.sh` con `touch mostrar-fecha.sh`, después, con `nano mostrar-fecha.sh` se ha modificado el script para que quede con el contenido de la siguiente imagen:

```
rafa@portatilrafa: ~/TEII/bloque2/contenedor1
GNU nano 8.3 mostrar-fecha.sh
#!/usr/bin/env bash
while true
do
    date
    sleep 10
done
```

Las órdenes usadas han sido:

```
(base) rafa@portatilrafa:~/TEII/bloque2$ mkdir contenedor1
(base) rafa@portatilrafa:~/TEII/bloque2$ cd contenedor1/
(base) rafa@portatilrafa:~/TEII/bloque2/contenedor1$ touch mostrar-fecha.sh
(base) rafa@portatilrafa:~/TEII/bloque2/contenedor1$ nano mostrar-fecha.sh
```

Ahora, se crea el `dockerfile` con `nano Dockerfile` y se le deja el contenido de la siguiente imagen:

```
rafa@portatilrafa: ~/TEII/bloque2/contenedor1
GNU nano 8.3 Dockerfile *
FROM ubuntu:latest
COPY mostrar-fecha.sh /mostrar-fecha.sh
CMD ["/mostrar-fecha.sh"]
```

Tras esto, se construye la imagen Docker con `docker build -t imagen1 .`

```
rafa@portatilrafa: ~/TEII/bloque2/contenedor1
(base) rafa@portatilrafa:~/TEII/bloque2/contenedor1$ docker build -t imagen1 .
[+] Building 0.7s (7/7) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 122B
=> [internal] load metadata for docker.io/library/ubuntu:latest
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 37B
=> CACHED [1/2] FROM docker.io/library/ubuntu:latest@sha256:186072bba1b2f436cbb91ef2567abca677337cfc786c86e107d25b7072feef0c
=> [2/2] COPY mostrar-fecha.sh /mostrar-fecha.sh
=> exporting to image
=> => exporting layers
=> => writing image sha256:20e176a5e77363fc7a2e537b52f8ae89ce69508fcdd7de32c73ab771a45f7690
=> => naming to docker.io/library/imagen1
```

Se puede comprobar que la imagen se ha creado con `docker image ls`

```
rafa@portatilrafa: ~/TEII/bloque2/contenedor1
(base) rafa@portatilrafa:~/TEII/bloque2/contenedor1$ docker image ls
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
aoc-prac-2023:latest	2ae650f24cd8	11.5GB	0B	
hello-world:latest	1b44b5a3e06a	10.1kB	0B	U
imagen1:latest	20e176a5e773	78.1MB	0B	

Ahora, se ejecuta el contenedor con *docker run --name contenedor1 imagen1*

```
rafa@portatilrafa: ~/TEII/bloque2/contenedor1
(base) rafa@portatilrafa:~/TEII/bloque2/contenedor1$ docker run --name contenedor1 imagen1:latest
Fri Mar 27 13:15:37 UTC 2026
Fri Mar 27 13:15:47 UTC 2026
```

Y como se puede observar, muestra la fecha y hora actual cada 10 segundos.

Ahora, el contenedor se para con *docker stop contenedor1* desde una nueva terminal.

• Segundo Contenedor

Primero, se crea un directorio (*mkdir contenedor2*) para este segundo contenedor y, dentro de este nuevo directorio (*cd contenedor2/*) se crea una carpeta donde se guardará el fichero (*mkdir salida*).

```
rafa@portatilrafa: ~/TEII/bloque2/contenedor2
(base) rafa@portatilrafa:~/TEII/bloque2$ mkdir contenedor2
(base) rafa@portatilrafa:~/TEII/bloque2$ cd contenedor2/
(base) rafa@portatilrafa:~/TEII/bloque2/contenedor2$ mkdir salida
```

Ahora, se crea un script *guardar-fecha.sh* (*touch guardar-fecha.sh*) y se le añade el contenido (*nano guardar-fecha.sh*) que se puede ver en la siguiente imagen:

```
rafa@portatilrafa: ~/TEII/bloque2/contenedor2
GNU nano 8.3                                guardar-fecha.sh
#!/usr/bin/env bash
while true
do
    date >> /fichero/fecha.txt
    sleep 60
done
```

Además, se le añaden permisos de ejecución con *chmod +x guardar-fecha.sh*

Ahora, se crea el Dockerfile (*touch Dockerfile*) y se le añade el contenido (*nano Dockerfile*) que se aprecia en la imagen a continuación:

```
rafa@portatilrafa: ~/TEII/bloque2/contenedor2
GNU nano 8.3                                Dockerfile
FROM ubuntu:latest
COPY guardar-fecha.sh /guardar-fecha.sh
CMD ["/guardar-fecha.sh"]
```

Tras esto, se construye la imagen con *docker build -t imagen2*.

```
rafa@portatilrafa: ~/TEII/bloque2/contenedor2
(base) rafa@portatilrafa:~/TEII/bloque2/contenedor2$ docker build -t imagen2 .
[+] Building 0.8s (7/7) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 122B
=> [internal] load metadata for docker.io/library/ubuntu:latest
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 119B
=> CACHED [1/2] FROM docker.io/library/ubuntu:latest@sha256:186072bba1b2f436cbb91ef2567abca677337cfc786c86e107d25b7072feef0c
=> [2/2] COPY guardar-fecha.sh /guardar-fecha.sh
=> exporting to image
=> => exporting layers
=> => writing image sha256:90347bcf4a70295581ce171e849761752114248c006f65dc219a418efa71d7f1
=> => naming to docker.io/library/imagen2
```

Se puede comprobar que la imagen se ha creado con *docker image ls*

```
rafa@portatilrafa: ~/TEII/bloque2/contenedor2
(base) rafa@portatilrafa:~/TEII/bloque2/contenedor2$ docker image ls

IMAGE                ID                DISK USAGE    CONTENT SIZE    EXTRA
aoc-prac-2023:latest 2ae650f24cd8      11.5GB        0B              U
hello-world:latest   1b44b5a3e06a      10.1kB        0B              U
imagen1:latest        20e176a5e773      78.1MB        0B
imagen2:latest        90347bcf4a70      78.1MB        0B
```

Ahora, se ejecuta el contenedor montando la carpeta salida de la máquina principal en /fichero dentro del contenedor con *docker run --name contenedor2 -v /home/rafa/TEII/bloque2/contenedor2/salida:/fichero imagen2*

```
rafa@portatilrafa: ~/TEII/bloque2/contenedor2
(base) rafa@portatilrafa:~/TEII/bloque2/contenedor2$ docker run --name contenedor2 -v /home/rafa/TEII/bloque2/contenedor2/salida:/fichero imagen2
```

Desde una nueva terminal, se puede comprobar que el fichero se va a actualizando añadiendo la fecha y hora actual cada minuto:

```
rafa@portatilrafa: ~/TEII/bloque2/contenedor2/salida
(base) rafa@portatilrafa:~/TEII/bloque2/contenedor2$ cd salida/
(base) rafa@portatilrafa:~/TEII/bloque2/contenedor2/salida$ cat fecha.txt
Fri Mar 27 14:08:42 UTC 2026
(base) rafa@portatilrafa:~/TEII/bloque2/contenedor2/salida$ cat fecha.txt
Fri Mar 27 14:08:42 UTC 2026
Fri Mar 27 14:09:42 UTC 2026
```

Por último, se detiene el contenedor con *docker stop contenedor2*

• Tercer Contenedor

Primero, se crea un directorio para este tercer contenedor (*mkdir contenedor3*) y dentro de este directorio (*cd contenedor3/*), se crea el fichero Dockerfile (*touch Dockerfile*) y se le añade el contenido (*nano Dockerfile*) que se puede ver en la siguiente imagen:

```
rafa@portatilrafa: ~/TEII/bloque2/contenedor3
GNU nano 8.3 Dockerfile
FROM nginx:latest
EXPOSE 80
```

De esta forma, se usa la imagen base *nginx:latest* que ya incorpora el servidor web, y se utiliza *EXPOSE* para señalar que el puerto 80 se exportará al ejecutar la imagen.

Entonces, ahora se construye la imagen con *docker build -t imagen3 .*

```
rafa@portatilrafa: ~/TEII/bloque2/contenedor3
(base) rafa@portatilrafa:~/TEII/bloque2/contenedor3$ docker build -t imagen3 .
[+] Building 5.2s (5/s) FINISHED
=> [internal] load build definition from Dockerfile
=> [internal] load metadata for docker.io/library/nginx:latest
=> [internal] load .dockerignore
=> transferring context: 2B
=> [1/1] FROM docker.io/library/nginx:latest@sha256:7150b3a39203cb50ee6327f4a5d18774f0c7ca63990ee8985e97e2eb751c10
=> resolve docker.io/library/nginx:latest@sha256:7150b3a39203cb50ee6327f4a5d18774f0c7ca63990ee8985e97e2eb751c10
=> sha256:8c71d6af3ca7262ca19a7f0db6c6d96f1410d3c1d078210787cf99032a217 9.09kB / 9.09kB
=> sha256:7150b3a39203cb50ee6327f4a5d18774f0c7ca63990ee8985e97e2eb751c10 18.23kB / 18.23kB
=> sha256:ec781dee3f8719c2ca809b73cb1d4e53ee1406c95a6e44b0ffaa5d1f87 29.70kB / 29.70kB
=> sha256:bb30aa29654655a18d97885cd3947d19ca516dd44c3341acc1bbc8d14a7a36 33.16kB / 33.16kB
=> sha256:510dd65570e18d540b6f7688a8d4fa925fea17316d335264eb3f092b8858cd0 626B / 626B
=> sha256:c3fe1eeaa810f4a585961f17339c93f87b1c7cd5c02c9101814752bd051961c 2.29kB / 2.29kB
=> sha256:cde7a85ae42831ee51be8948b88b25c297a1088875a3479c55a65f1e620fdd73 955B / 955B
=> extracting sha256:ec781dee3f8719c2ca809b73cb1d4e53ee1406c95a6e44b0ffaa5d1f87
=> sha256:587e3d8dbdb56fc406b2b292318c9a46e72c144ad849b5ef8755f581e6704 402B / 402B
=> sha256:318808c8df8a244f1766a8d1971de41588b784226a040aefc76eeeb37d99 1.21kB / 1.21kB
=> sha256:5a812e87569b9047921446a2a390647495d99c08f253a082f528f4814469ef 9.40kB / 9.40kB
=> extracting sha256:bb30aa29654655a18d97885cd3947d19ca516dd44c3341acc1bbc8d14a7a36
=> extracting sha256:510dd65570e18d540b6f7688a8d4fa925fea17316d335264eb3f092b8858cd0
=> extracting sha256:cde7a85ae42831ee51be8948b88b25c297a1088875a3479c55a65f1e620fdd73
=> extracting sha256:587e3d8dbdb56fc406b2b292318c9a46e72c144ad849b5ef8755f581e6704
=> extracting sha256:318808c8df8a244f1766a8d1971de41588b784226a040aefc76eeeb37d99
=> extracting sha256:5a812e87569b904847921446a2a390647495d99c08f253a082f528f4814469ef
=> exporting layers
=> writing image sha256:742ebaf8dfb9f2cfa9a3cb7d819b7528ceb8b188881c8db1feb6313aceea77
=> naming to docker.io/library/imagen3
```

Se puede comprobar que se ha creado correctamente con *docker image ls*

```
rafa@portatilrafa: ~/TEII/bloque2/contenedor3
(base) rafa@portatilrafa:~/TEII/bloque2/contenedor3$ docker image ls
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
aoc-prac-2023:latest	2ae650f24cd8	11.5GB	0B	
hello-world:latest	1b44b5a3e06a	10.1kB	0B	U
imagen1:latest	20e176a5e773	78.1MB	0B	U
imagen2:latest	1ee28d9a457f	78.1MB	0B	U
imagen3:latest	742ebaf6dfb9	161MB	0B	

Ahora, se ejecuta el contenedor publicando el servicio web y montando en él la carpeta en la que el segundo contenedor guarda el fichero fecha.txt. Para esto se utiliza el comando *docker run -d --name contenedor3 -p 8080:80 -v /home/rafa/TEII/bloque2/contenedor2/salida:/usr/share/nginx/html imagen3*

```
rafa@portatilrafa: ~/TEII/bloque2/contenedor3
(base) rafa@portatilrafa:~/TEII/bloque2/contenedor3$ docker run -d --name contenedor3 -p 8080:80 -v /home/rafa/TEII/bloque2/contenedor2/salida:/usr/share/nginx/html imagen3
5993a77be0a2a26cd57d70b9efb2181f62eb4a87868164181cfa5ef9c5bdced6
```

Tras esto, se comprueba desde el host que se muestra el fichero generado por el segundo contenedor mediante *curl http://127.0.0.1:8080/fecha.txt*

```
(base) rafa@portatilrafa:~/TEII/bloque2/contenedor3$ curl http://127.0.0.1:8080/fecha.txt
Fri Mar 27 14:08:42 UTC 2026
Fri Mar 27 14:09:42 UTC 2026
Fri Mar 27 14:10:42 UTC 2026
Fri Mar 27 14:11:42 UTC 2026
Fri Mar 27 14:12:42 UTC 2026
Fri Mar 27 14:13:42 UTC 2026
Fri Mar 27 14:14:42 UTC 2026
Fri Mar 27 14:15:42 UTC 2026
```

Se puede observar que funciona correctamente.

Para detener el contenedor se utiliza *docker stop contenedor3*

- **Docker-compose**

Primero, se crea un directorio (*mkdir practica-docker-compose*) y se entra en él (*cd practica-docker-compose/*). Ahora se crean tres subdirectorios, uno para cada imagen:

- *mkdir contenedor1*
- *mkdir contenedor2*
- *mkdir contenedor3*

```
rafa@portatilrafa: ~/TEII/bloque2/practica-docker-compose
(base) rafa@portatilrafa:~/TEII/bloque2$ mkdir practica-docker-compose
(base) rafa@portatilrafa:~/TEII/bloque2$ cd practica-docker-compose/
(base) rafa@portatilrafa:~/TEII/bloque2/practica-docker-compose$ mkdir contenedor1
(base) rafa@portatilrafa:~/TEII/bloque2/practica-docker-compose$ mkdir contenedor2
(base) rafa@portatilrafa:~/TEII/bloque2/practica-docker-compose$ mkdir contenedor3
```

Ahora, para cada directorio de cada contenedor, se copia el contenido que se ha creado en los apartados anteriores:

- Para el contenedor1: el script mostrar-fecha.sh y el Dockerfile.
- Para el contenedor2: el script guardar-fecha.sh y el Dockerfile.
- Para el contenedor3: el Dockerfile.

Debe quedar así:

```
rafa@portatilrafa: ~/TEII/bloque2/practica-docker-compose
(base) rafa@portatilrafa:~/TEII/bloque2/practica-docker-compose$ ls -l contenedor*
contenedor1:
total 8
-rw-rw-r-- 1 rafa rafa 85 mar 27 14:13 Dockerfile
-rwxrwxr-x 1 rafa rafa 55 mar 27 13:43 mostrar-fecha.sh

contenedor2:
total 8
-rw-rw-r-- 1 rafa rafa 85 mar 27 14:53 Dockerfile
-rwxrwxr-x 1 rafa rafa 77 mar 27 15:06 guardar-fecha.sh

contenedor3:
total 4
-rw-rw-r-- 1 rafa rafa 28 mar 27 15:47 Dockerfile
```

Ahora, en el directorio principal (practica-docker-compose) se crea el fichero “docker-compose.yml” (*touch docker-compose.yml*) y se le añade el contenido (*nano docker-compose.yml*) de la siguiente imagen:

```

GNU nano 8.3                                     docker-compose.yml
version: "3"

services:
  contenedor1:
    build: ./contenedor1

  contenedor2:
    build: ./contenedor2
    volumes:
      - fichero-compartido:/fichero

  contenedor3:
    build: ./contenedor3
    ports:
      - "8080:80"
    volumes:
      - fichero-compartido:/usr/share/nginx/html
    depends_on:
      - contenedor2

volumes:
  fichero-compartido:

```

Tras esto, se levanta el servicio completo con *docker-compose up -d*

```
(base) rafa@portatilrafa: ~/TEII/bloque2/practica-docker-compose$ docker-compose up -d
WARN[0000] /home/rafa/TEII/bloque2/practica-docker-compose/docker-compose.yml: the attribute `version` is obsolete, it will be removed in a future release. To prevent this warning, you have three options:
#1 [internal] load local bake definitions
#1 reading from stdin 1.70kB done
#1 DONE 0.0s

#2 [contenedor1 internal] load build definition from Dockerfile
#2 transferring dockerfile: 122B done
#2 DONE 0.0s

#3 [contenedor3 internal] load build definition from Dockerfile
#3 transferring dockerfile: 65B done
#3 DONE 0.0s

#4 [contenedor2 internal] load build definition from Dockerfile
#4 transferring dockerfile: 122B done
```

```
✓ practica-docker-compose-contenedor3 Built
✓ practica-docker-compose-contenedor1 Built
✓ practica-docker-compose-contenedor2 Built
✓ Network practica-docker-compose_default Created
✓ Volume "practica-docker-compose_fichero-compartido" Created
✓ Container practica-docker-compose-contenedor1-1 Started
✓ Container practica-docker-compose-contenedor2-1 Started
✓ Container practica-docker-compose-contenedor3-1 Started
```

Ahora, desde el host se hace `curl http://127.0.0.1:8080/fecha.txt`

```
rafa@portatilrafa: ~/TEII/bloque2/practica-docker-compose
(base) rafa@portatilrafa:~/TEII/bloque2/practica-docker-compose$ curl http://127.0.0.1:8080/fecha.txt
Fri Mar 27 15:20:14 UTC 2026
Fri Mar 27 15:21:14 UTC 2026
Fri Mar 27 15:22:14 UTC 2026
```

Como se puede comprobar, funciona correctamente.

Por último, se hace `docker-compose down` para eliminar los contenedores:

```
rafa@portatilrafa: ~/TEII/bloque2/practica-docker-compose
(base) rafa@portatilrafa:~/TEII/bloque2/practica-docker-compose$ docker-compose down
WARN[0000] /home/rafa/TEII/bloque2/practica-docker-compose/docker-compose.yml: the attribute 'version' is obsolete, it will be ignored, please remove it to avoid potential confusion
[+] Running 4/4
✔ Container practica-docker-compose-contenedor1-1 Removed 10s
✔ Container practica-docker-compose-contenedor3-1 Removed 0s
✔ Container practica-docker-compose-contenedor2-1 Removed 10s
✔ Network practica-docker-compose_default Removed 0s
```